Rapport de projet

Calcul scientifique et Modélisation

Marc ROBIN – Théau RECZULSKI

Table des matières

Chapitre I.
Réponses aux questions théoriques

1. Préliminaires

Notation : Sauf mention du contraire, la norme $||\cdot||$ sans indice correspond à la norme euclidienne sur $\mathbb{R}^2$.

Question 1

Soit $v \in V$. En multipliant l'équation (1) par v et en intégrant sur Ω, on obtient :

$$\int_\Omega f v \, dxdy = \int_\Omega \left(-v \frac{\partial}{\partial x}\left(\eta \frac{\partial u}{\partial x}\right) - v \frac{\partial}{\partial y}\left(\eta \frac{\partial u}{\partial y}\right) + \lambda u v \right) dxdy$$

$$= \int_\Omega \lambda u v \, dxdy + \int_{-b}^{b}\int_{-a}^{a} -v \frac{\partial}{\partial x}\left(\eta \frac{\partial u}{\partial x}\right) dxdy + \int_{-a}^{a}\int_{-b}^{b} -v \frac{\partial}{\partial y}\left(\eta \frac{\partial u}{\partial y}\right) dydx.$$

En appliquant une intégration par parties sur les termes avec les dérivées partielles :

$$\int_\Omega f v \, dxdy = \int_\Omega \lambda u v \, dxdy$$

$$+ \int_{-b}^{b} \left(\left[-v\eta \frac{\partial u}{\partial x} \right]_{-a}^{a} + \int_{-a}^{a} \frac{\partial v}{\partial x} \eta \frac{\partial u}{\partial x} \, dx \right) dy$$

$$+ \int_{-a}^{a} \left(\left[-v\eta \frac{\partial u}{\partial y} \right]_{-b}^{b} + \int_{-b}^{b} \frac{\partial v}{\partial y} \eta \frac{\partial u}{\partial y} \, dy \right) dx$$

D'après les conditions aux limites, les dérivées normales s'annulent au bord, donc :

$$\left[-v\eta\frac{\partial u}{\partial x}\right]_{-a}^{a} = 0 \quad \text{et} \quad \left[-v\eta\frac{\partial u}{\partial y}\right]_{-b}^{b} = 0.$$

Finalement, nous avons l'égalité suivante :

$$\begin{aligned}\int_\Omega fv\,dxdy &= \int_\Omega \lambda uv\,dxdy + \int_\Omega \eta\frac{\partial v}{\partial x}\frac{\partial u}{\partial x}\,dxdy + \int_\Omega \eta\frac{\partial v}{\partial y}\frac{\partial u}{\partial y}\,dxdy \\ &= \int_\Omega \left(\eta\,\langle\,\nabla v\,|\,\nabla u\,\rangle + \lambda uv\right)\,dxdy.\end{aligned}$$

Question 2

Soit $w \in V$ vérifiant :

$$\int_\Omega fv\,dxdy = \int_\Omega \left(\eta\nabla v\cdot\nabla u + \lambda uv\right)\,dxdy.$$

Comme u vérifie la même équation, on a :

$$\int_\Omega \eta\,\langle\,\nabla w - \nabla u\,|\,\nabla v\,\rangle\, + \lambda v(w-u)\,dxdy = 0.$$

Ceci étant vrai pour tout $v \in V$, en particulier, c'est vrai pour $v = w - u \in V$. On obtient alors :

$$\int_\Omega \eta\,\langle\,\nabla w - \nabla u\,|\,\nabla w - \nabla u\,\rangle\, + \lambda(w-u)^2\,dxdy = 0$$

$$\iff \quad \int_\Omega \eta\,||\,\nabla w - \nabla u\,||^2 + \lambda(w-u)^2\,dxdy = 0.$$

Or $\eta\,||\,\nabla w - \nabla u\,||^2 + \lambda(w-u)^2 \geq 0$ car $\eta \geq \eta_0 > 0$ et $\lambda > 0$. On en déduit donc que

$$\int_\Omega \eta\,||\,\nabla w - \nabla u\,||^2 + \lambda(w-u)^2\,dxdy = 0 \quad \iff \quad w = u.$$

Question 3

En supposant $\eta = 1$, $\lambda = 1$, on a :

$$-\frac{\partial}{\partial x}\left(\eta\frac{\partial u}{\partial x}\right) - \frac{\partial}{\partial y}\left(\eta\frac{\partial u}{\partial y}\right) + \lambda u = f(x,y)$$

$$\iff \quad -\frac{\partial^2 u}{\partial x^2} - \frac{\partial^2 u}{\partial y^2} + u = f(x,y)$$

$$\iff \quad -\frac{\partial^2 u}{\partial x^2} - \frac{\partial^2 u}{\partial y^2} + u = \cos(m\pi x)\cos(m\pi y).$$

Posons $u(x,y) = C\cos(m\pi x)\cos(m\pi y) \quad \forall\,(x,y) \in \overline{\Omega}$. Alors

$$\frac{\partial u}{\partial x}(x,y) = -C\sin(m\pi x)\cos(m\pi y)$$

et

$$\frac{\partial u}{\partial y}(x,y) = -C\cos(m\pi x)\sin(m\pi y).$$

On remarque que u remplit bien les conditions aux bords si $a = b = 1$:

$$\frac{\partial u}{\partial x}(\pm 1, y) = \frac{\partial u}{\partial y}(x, \pm 1) = 0.$$

Il ne nous reste donc plus qu'à déterminer C. On a :

$$\frac{\partial^2 u}{\partial x^2}(x,y) = \frac{\partial^2 u}{\partial y^2}(x,y) = -Cm^2\pi^2\cos(m\pi x)\cos(m\pi y).$$

Ainsi,

$$-\frac{\partial^2 u}{\partial x^2}(x,y) - \frac{\partial^2 u}{\partial y^2}(x,y) + u(x,y) = \cos(m\pi x)\cos(m\pi y)$$

$$\iff \quad C(m^2\pi^2 + m^2\pi^2 + 1)\cos(m\pi x)\cos(m\pi y) = \cos(m\pi x)\cos(m\pi y)$$

$$\iff \quad C = \frac{1}{2m^2\pi^2 + 1}.$$

Par suite, on a

$$u^p(x,y) = \frac{\cos(m\pi x)\cos(m\pi y)}{2m^2\pi^2 + 1}$$

Question 4

Théorème. *Changement de variable*
Soit $n \in \mathbb{N}$, $\mathcal{C}$ et $\mathcal{D}$ deux ouverts de $\mathbb{R}^n$ et $\phi : \mathcal{C} \to \mathcal{D}$ un C^1-difféomorphisme.
Si $f : \mathcal{D} \to \mathbb{R}$ est borélienne positive ou intégrable, alors

$$\int_{\mathcal{D}} f \, d\lambda^n = \int_{\mathcal{C}} (f \circ \phi) \, |\det D\phi| \, d\lambda^n$$

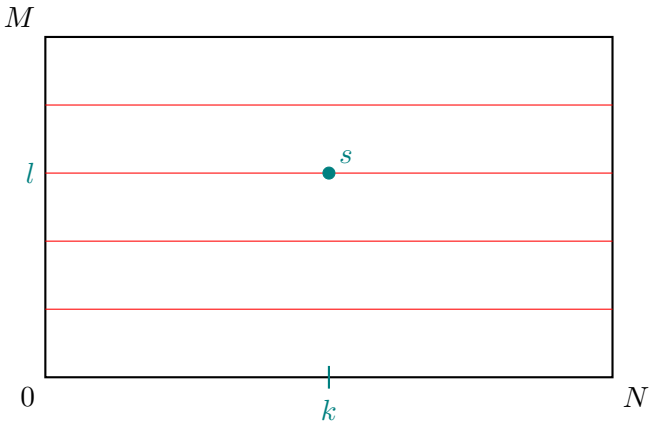
2. Le maillage

2.1 Les noeuds

Remarque.
Les fonctions des questions 5, 7, 9, 11, 22, 28(b) et 28(c) sont incluses dans le code fourni de l'annexe B.

Numérotation de tous les noeuds

Question 6



On a $s = l(N+1)+k$ où $l(N+1)$ représente le nombre de points sur les l premières lignes et k représente le nombre de points que l'on compte sur la $(l+1)$-ème ligne pour arriver à s.

Question 8

Soit $s \in \{0, \cdots, (N+1)(M+1)-1\}$. On effectue la division euclidienne de s par $N+1\,(>0)$. On obtient alors :

$$s = (N+1)\,q + r, \quad \text{avec} \quad r < N+1.$$

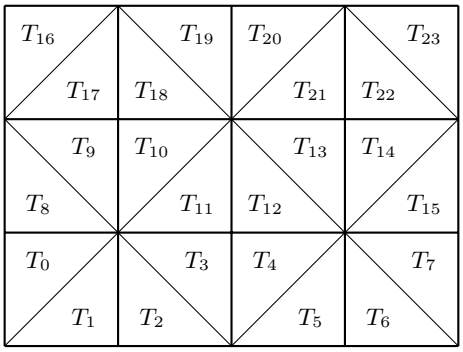
De plus, $s < (N+1)(M+1)$ donc $q < M+1$. En posant $j = q$ et $r = i$, on a l'existence. L'unicité vient de l'unicité de la division euclidienne.

2.2 La triangulation

Une partition en triangles

Question 10

Pour illustrer la situation, nous avons décidé de poser $N = 4$ et $M = 3$, pour un total de $K = 24$ triangles.



Coordonnées barycentriques dans un triangle. Triangle de référence

Question 12

Soit $g : x \longmapsto f(x) - f(0)$ une application de $\mathbb{R}^n$ dans $\mathbb{R}^m$. Supposons que f est affine. Alors,

$$\forall\, x, y \in \mathbb{R}^n, \quad \forall\, \alpha \in \mathbb{R}, \quad f(\alpha x + (1-\alpha)\, y) = \alpha f(x) + (1-\alpha)\, f(y).$$

Alors d'une part, si $\lambda \in \mathbb{R}$,

$$\begin{aligned} g(\lambda x) &= f(\lambda x + (1-\lambda)\, 0) - f(0) \\ &= \lambda f(x) + (1-\lambda) f(0) \\ &= \lambda\, (f(x) - f(0)) = \lambda g(x), \end{aligned}$$

et d'autre part,

$$\begin{aligned} g(x+y) &= 2g\left(\frac{1}{2}x + \frac{1}{2}y\right) \\ &= 2\left(\frac{1}{2}f(x) + \frac{1}{2}f(y) - f(0)\right) \\ &= f(x) + f(y) - f(0) = g(x) + g(y). \end{aligned}$$

Par suite, g est linéaire. Réciproquement, supposons que g est linéaire. Comme $g(x) = f(x) - f(0)$, on en déduit que $f(x) = g(x) - f(0)$. Ainsi,

$$\begin{aligned} f(\alpha x + (1-\alpha)\, y) =& g(\alpha x + (1-\alpha)\, y) + f(0) \\ &= \alpha g(x) + (1-\alpha)\, g(y) + f(0) \\ &= \alpha(f(x) - f(0)) + f(y) - f(0) - \alpha(f(y) - f(0)) + f(0) \\ &= \alpha f(x) + (1-\alpha)\, f(y). \end{aligned}$$

f est donc affine et cela permet de conclure.

Question 13

Comme vu à la question précédente, $f : \mathbb{R}^2 \longrightarrow \mathbb{R}$ est affine si et seulement si

$$g : \mathbb{R}^2 \longrightarrow \mathbb{R}, \quad (x, y) \longmapsto f(x, y) - f(0, 0)$$

est linéaire. Si g est une forme linéaire, alors elle s'écrit avec une matrice ligne de taille 1×2. On en déduit directement qu'il existe α, β deux réels tels que

$$\forall\, (x, y) \in \mathbb{R}^2, \quad g(x, y) = \alpha x + \beta y.$$

On a donc

$$g(x, y) = f(x, y) - f(0, 0) \quad \Longleftrightarrow \quad \alpha x + \beta y = f(x, y) - f(0, 0).$$

Finalement, on a bien $f(x, y) = \alpha x + \beta y + \gamma$, où $\gamma = f(0, 0)$.

Question 14

Puisque A_0, A_1 et A_2 ne sont pas alignés, les vecteurs $\overrightarrow{A_0A_1}$ et $\overrightarrow{A_0A_2}$ forment une base de $\mathbb{R}^2$. Par suite, le vecteur $\overrightarrow{A_0M}$ se décompose de manière unique dans cette base :

$$\overrightarrow{A_0M} = \alpha \overrightarrow{A_0A_1} + \beta \overrightarrow{A_0A_2}.$$

On en déduit que

$$
\begin{aligned}
M = A_0 + \overrightarrow{A_0M} \quad \Longleftrightarrow \quad M - A_0 &= \overrightarrow{A_0M} \\
&= \alpha \overrightarrow{A_0A_1} + \beta \overrightarrow{A_0A_2} \\
&= \alpha\left(A_1 - A_0\right) + \beta\left(A_2 - A_0\right) \\
&= \left(-\alpha - \beta\right) A_0 + \alpha A_1 + \beta A_2.
\end{aligned}
$$

On a donc finalement que $M = (1 - \alpha - \beta)\, A_0 + \alpha A_1 + \beta A_2$. On en déduit alors que

$$
\lambda_0(x, y) = 1 - \alpha - \beta, \quad \lambda_1(x, y) = \alpha, \quad \lambda_2(x, y) = \beta
$$

et leur somme vaut bien 1.

Question 15

Soit $f_i : (x, y) \longmapsto \lambda_i(x, y)$. On veut montrer que f_i est de la forme $\alpha x + \beta y + \gamma$. D'après la question 14, on a le système

$$
\begin{cases}
x = \lambda_0 x_0 + \lambda_1 x_1 + \lambda_2 x_2 \\
y = \lambda_0 y_0 + \lambda_1 y_1 + \lambda_2 y_2 \\
1 = \lambda_0 + \lambda_1 + \lambda_2
\end{cases}
$$

qui se traduit par le système matriciel suivant :

$$
\underbrace{\begin{pmatrix} x_0 & x_1 & x_2 \\ y_0 & y_1 & y_2 \\ 1 & 1 & 1 \end{pmatrix}}_{A} \begin{pmatrix} \lambda_0 \\ \lambda_1 \\ \lambda_2 \end{pmatrix} = \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}.
$$

De plus,

$$
\det(A) = \begin{vmatrix} x_0 & x_1 & x_2 \\ y_0 & y_1 & y_2 \\ 1 & 1 & 1 \end{vmatrix} = \begin{vmatrix} x_0 & x_1 - x_0 & x_2 - x_0 \\ y_0 & y_1 - y_0 & y_2 - y_0 \\ 1 & 0 & 0 \end{vmatrix} = \begin{vmatrix} x_1 - x_0 & x_2 - x_0 \\ y_1 - y_0 & y_2 - y_0 \end{vmatrix}.
$$

Comme

$$
\begin{pmatrix} x_1 - x_0 \\ y_1 - y_0 \end{pmatrix} = \overrightarrow{A_0A_1}.
$$

et

$$
\begin{pmatrix} x_2 - x_0 \\ y_2 - y_0 \end{pmatrix} = \overrightarrow{A_0A_2}
$$

et que $\overrightarrow{A_0A_1}$ et $\overrightarrow{A_0A_2}$ forment une base de $\mathbb{R}^2$, on en déduit que A est inversible. Ainsi,

$$\begin{pmatrix} \lambda_0 \\ \lambda_1 \\ \lambda_2 \end{pmatrix} = A^{-1} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

.
Finalement,

$$\lambda_i = \alpha_i x_i + \beta_i y_i + \gamma_i.$$

On conclut grâce à la question 13 que f_i est affine.

Question 16

Soit $i \in \{0, 1, 2\}$. On a

$$\begin{pmatrix} x_i \\ y_i \end{pmatrix} = \delta_{0,i} \begin{pmatrix} x_0 \\ y_0 \end{pmatrix} + \delta_{1,i} \begin{pmatrix} x_1 \\ y_1 \end{pmatrix} + \delta_{2,i} \begin{pmatrix} x_2 \\ y_2 \end{pmatrix}.$$

On en déduit directement que $\lambda_j(x_i, y_i) = \delta_{i,j}$.

Question 17

Soit f une application affine de $\mathbb{R}^2$ dans $\mathbb{R}$. Soient $\alpha, \beta, \gamma \in \mathbb{R}$ tels que pour tout $(x, y) \in \mathbb{R}^2$,

$$f(x, y) = \alpha x + \beta y + \gamma.$$

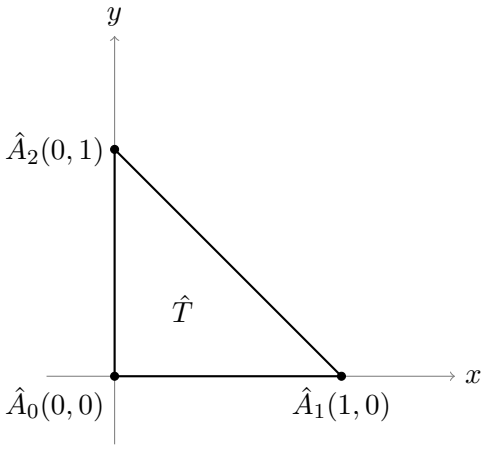
D'après la question 14, on a :

$$\begin{aligned} f(x, y) &= \alpha\left(\lambda_0 x_0 + \lambda_1 x_1 + \lambda_2 x_2\right) + \beta\left(\lambda_0 y_0 + \lambda_1 y_1 + \lambda_2 y_2 + \gamma\left(\lambda_0 + \lambda_1 + \lambda_2\right)\right. \\ &= \lambda_0\left(\alpha x_0 + \beta y_0 + \gamma\right) + \lambda_1\left(\alpha x_1 + \beta y_1 + \gamma\right) + \lambda_2\left(\alpha x_2 + \beta y_2 + \gamma\right) \\ &= \sum_{i=0}^{2} f(x_i, y_i)\lambda_i. \end{aligned}$$

Si pour tout $i \in \{0, 1, 2\}$, $f(x_i, y_i) = 0$, alors

$$f(x, y) = \sum_{i=0}^{2} 0 \cdot \lambda_i(x, y) = 0 \quad \text{pour tout } (x, y) \in \mathbb{R}^2.$$

Question 18



Soit $(x, y) \in \mathbb{R}^2$. Alors de

$$\begin{pmatrix} x \\ y \end{pmatrix} = \hat{\lambda_0} \begin{pmatrix} 0 \\ 0 \end{pmatrix} + \hat{\lambda_1} \begin{pmatrix} 1 \\ 0 \end{pmatrix} + \hat{\lambda_2} \begin{pmatrix} 0 \\ 1 \end{pmatrix},$$

on déduit que $\hat{\lambda_1} = x$ et $\hat{\lambda_2} = y$. Comme de plus la somme des $\hat{\lambda_i}$ fait 1, alors $\hat{\lambda_0} = 1 - x - y$.

Question 19

Posons l'application $F_T : \mathbb{R}^2 \to \mathbb{R}^2$ définie par :

$$\begin{aligned} F_T(\hat{x}, \hat{y}) &= \hat{\lambda}_0(\hat{x}, \hat{y})A_0 + \hat{\lambda}_1(\hat{x}, \hat{y})A_1 + \hat{\lambda}_2(\hat{x}, \hat{y})A_2 \\ &= (1 - \hat{x} - \hat{y})A_0 + \hat{x}A_1 + \hat{y}A_2 \\ &= A_0 + \hat{x}(A_1 - A_0) + \hat{y}(A_2 - A_0). \end{aligned}$$

où les A_i sont les sommets du triangle T. On reconnaît que F_T s'écrit comme la somme d'un vecteur constant A_0 et d'une combinaison linéaire des variables $(\hat{x}, \hat{y})$ avec les vecteurs constants $\overrightarrow{A_0A_1}$ et $\overrightarrow{A_0A_2}$. L'application F_T est donc bien une transformation affine. De plus, elle transforme bien les sommets de $\hat{T}$ en ceux de T. En effet, D'après la question 16, on sait que $\hat{\lambda}_i(\hat{A}_j) = \delta_{ij}$ (où δ_{ij} vaut 1 si $i = j$ et 0 sinon). On a donc :

$$F_T(\hat{A}_j) = \sum_{i=0}^{2} \hat{\lambda}_i(\hat{A}_j)A_i = A_j$$

Il s'agit désormais de montrer l'unicité de F_T. Posons $F_T(\hat{x},\hat{y}) = \begin{pmatrix} f_1(\hat{x},\hat{y}) \\ f_2(\hat{x},\hat{y}) \end{pmatrix}$. Comme f_1 est une application de $\mathbb{R}^2 \longrightarrow \mathbb{R}$, alors par la question 17, elle est uniquement définie par les valeurs aux sommets de $\hat{T}$. De la même manière, f_2 est aussi uniquement définie par les valeurs aux sommets de $\hat{T}$. Les composantes f_1 et f_2 étant définies de façon unique par les coordonnées des sommets A_i, la transformation F_T est elle-même unique.

Question 20

Montrons que F_T est inversible. Posons $F_T(X) = B_T X + C$, où C est un vecteur de $\mathbb{R}^2$ et B_T une matrice 2×2 à coefficients réels qui représente la partie linéaire de F_T. Alors F_T est inversible si et seulement si B_T est inversible. Par définition de F_T, nous avons le système suivant :

$$\begin{cases} F_T(\hat{A}_0) = A_0 \\ F_T(\hat{A}_1) = A_1 \\ F_T(\hat{A}_2) = A_2 \end{cases}.$$

Dans le triangle $\hat{T}$, les points ont pour coordonnées $\hat{A}_0(0,0)$, $\hat{A}_1(1,0)$ et $\hat{A}_2(0,1)$. Les vecteurs $\overrightarrow{\hat{A}_0\hat{A}_1}$ et $\overrightarrow{\hat{A}_0\hat{A}_2}$ ont donc pour coordonnées respectives

$$\begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad \text{et} \quad \begin{pmatrix} 0 \\ 1 \end{pmatrix},$$

ce qui correspond à la base canonique de $\mathbb{R}^2$. Par linéarité de B_T, on obtient :

$$\begin{aligned} B_T(\overrightarrow{\hat{A}_0\hat{A}_1}) &= F_T(\hat{A}_0) - F_T(\hat{A}_1) = \overrightarrow{A_0A_1} \\ B_T(\overrightarrow{\hat{A}_0\hat{A}_2}) &= F_T(\hat{A}_0) - F_T(\hat{A}_2) = \overrightarrow{A_0A_2} \end{aligned}$$

.

Dans la base canonique, on a donc :

$$B_T = \begin{pmatrix} \overrightarrow{A_0A_1} & \overrightarrow{A_0A_2} \end{pmatrix}.$$

Or comme vu à la question 14, les vecteurs $\overrightarrow{A_0A_1}$ $\overrightarrow{A_0A_2}$ forment une base de $\mathbb{R}^2$. Par suite, B_T est inversible et donc F_T l'est également.

Question 21

Soit $F_T(\hat{x}, \hat{y}) = (f_1(\hat{x}, \hat{y}), f_2(\hat{x}, \hat{y}))$ la transformation affine de $\mathbb{R}^2$ dans $\mathbb{R}^2$ qui transforme le triangle de référence $\hat{T}$ en T. En utilisant les fonctions coordonnées barycentriques $\hat{\lambda}_0$, $\hat{\lambda}_1$ et $\hat{\lambda}_2$ du triangle de référence $\hat{T}$, on peut exprimer l'image d'un point par F_T comme combinaison affine des sommets $A_0(x_0, y_0)$, $A_1(x_1, y_1)$ et $A_2(x_2, y_2)$:

$$F_T(\hat{x}, \hat{y}) = \hat{\lambda}_0(\hat{x}, \hat{y})A_0 + \hat{\lambda}_1(\hat{x}, \hat{y})A_1 + \hat{\lambda}_2(\hat{x}, \hat{y})A_2$$

Comme $\hat{\lambda}_0(\hat{x}, \hat{y}) = 1 - \hat{x} - \hat{y}$, $\hat{\lambda}_1(\hat{x}, \hat{y}) = \hat{x}$ et $\hat{\lambda}_2(\hat{x}, \hat{y}) = \hat{y}$, on a :

$$F_T(\hat{x}, \hat{y}) = (1 - \hat{x} - \hat{y}) \begin{pmatrix} x_0 \\ y_0 \end{pmatrix} + \hat{x} \begin{pmatrix} x_1 \\ y_1 \end{pmatrix} + \hat{y} \begin{pmatrix} x_2 \\ y_2 \end{pmatrix}$$

En séparant les composantes f_1 et f_2, on obtient :

$$f_1(\hat{x}, \hat{y}) = x_0 + \hat{x}(x_1 - x_0) + \hat{y}(x_2 - x_0)$$

$$f_2(\hat{x}, \hat{y}) = y_0 + \hat{x}(y_1 - y_0) + \hat{y}(y_2 - y_0)$$

Par définition, la matrice jacobienne B_T est :

$$B_T = \begin{pmatrix} \frac{\partial f_1}{\partial \hat{x}} & \frac{\partial f_1}{\partial \hat{y}} \\ \frac{\partial f_2}{\partial \hat{x}} & \frac{\partial f_2}{\partial \hat{y}} \end{pmatrix}.$$

Les dérivées partielles sont :

$$\frac{\partial f_1}{\partial \hat{x}} = x_1 - x_0, \quad \frac{\partial f_1}{\partial \hat{y}} = x_2 - x_0, \quad \frac{\partial f_2}{\partial \hat{x}} = y_1 - y_0, \quad \frac{\partial f_2}{\partial \hat{y}} = y_2 - y_0.$$

La matrice jacobienne B_T en fonction des coordonnées des sommets du triangle T est donc :

$$B_T = \begin{pmatrix} x_1 - x_0 & x_2 - x_0 \\ y_1 - y_0 & y_2 - y_0 \end{pmatrix}$$

Question 23

Notons l'aire du triangle T $Aire(T)$. ALors par définition, on a

$$Aire(T) = \int_T 1 dxdy.$$

On effectue le changement de variable $(x, y) = F_T(\hat{x}, \hat{y})$. La formule du changement de variable nous donne :

$$Aire(T) = \int_{\hat{T}} |\det(B_T)| d\hat{x}d\hat{y}.$$

De plus, l'aire du triangle de référence, qui est rectangle isocèle, est donnée par

$$Aire(\hat{T}) = \frac{1 \times 1}{2} = \frac{1}{2}.$$

On a donc finalement

$$Aire(T) = \frac{1}{2}|\det(B_T)|.$$

Question 24

Soit M un point quelconque du triangle T. Ses coordonnées barycentriques par rapport aux sommets A_0, A_1, et A_2 vérifient :

$$M = \lambda_0(M)A_0 + \lambda_1(M)A_1 + \lambda_2(M)A_2,$$

où $\sum_{i=0}^{2} \lambda_i(M) = 1$. D'après la question 20, F_T est inversible. On en déduit donc que F_T^{-1} est une bijection affine qui envoie T sur $\hat{T}$. Ainsi,

$$\begin{aligned} F_T^{-1}(M) &= \lambda_0(M)F_T^{-1}(A_0) + \lambda_1(M)F_T^{-1}(A_1) + \lambda_2(M)F_T^{-1}(A_2) \\ &= \lambda_0(M)\hat{A}_0 + \lambda_1(M)\hat{A}_1 + \lambda_2(M)\hat{A}_2 \end{aligned}$$

Si on note $\hat{M} = F_T^{-1}(M) \in \hat{T}$, alors les $\lambda_i(M)$ sont les coordonnées barycentriques du point $\hat{M}$ par rapport au triangle de référence $\hat{T}$. Par unicité des coordonnées barycentriques, on a donc :

$$\lambda_i(M) = \hat{\lambda}_i(\hat{M}) = \hat{\lambda}_i(F_T^{-1}(\hat{M})).$$

On en déduit finalement que

$$\lambda_i = \hat{\lambda}_i \circ F_T^{-1}.$$

Question 25

D'après la question 24, $\lambda_i = \hat{\lambda}_i \circ F_T^{-1}$. On a donc que :

$$\mathrm{Jac}(\lambda_i) = \mathrm{Jac}(\hat{\lambda}_i) \times \mathrm{Jac}(F_T^{-1}),$$

où $\mathrm{Jac}(f) = (\nabla f)^T$ est la jacobienne de la fonction f. Comme de plus $\mathrm{Jac}(F_T^{-1}) = B_T^{-1}$ (par les questions 20 et 21), on a :

$$\begin{aligned} & \mathrm{Jac}(\lambda_i) = \mathrm{Jac}(\hat{\lambda}_i) \times \mathrm{Jac}(F_T^{-1}) \\ \iff \quad & (\nabla \lambda_i)^T = (\nabla \hat{\lambda}_i)^T B_T^{-1} \\ \iff \quad & \nabla \lambda_i = (B_T^{-1})^T \nabla \hat{\lambda}_i. \end{aligned}$$

Question 26

On cherche à calculer $\int_T \eta \langle \nabla \lambda_k \,|\, \nabla \lambda_j \rangle \, dxdy$. D'après la question 25, $\nabla \lambda_i = (B_T^{-1})^T \nabla \hat{\lambda}_i$. De plus, $\langle \nabla \lambda_k \,|\, \nabla \lambda_j \rangle$ est une constante. On a donc :

$$\int_T \eta \langle \nabla \lambda_k \,|\, \nabla \lambda_j \rangle \, dxdy = \langle (B_T^{-1})^T \nabla \hat{\lambda}_k \,|\, (B_T^{-1})^T \nabla \hat{\lambda}_j \rangle \int_T \eta \, dxdy.$$

Exprimons maintenant $\int_T \lambda_k \lambda_j \, dxdy$. En effectuant le changement de variables $(x, y) = F_T(\hat{x}, \hat{y})$ et en réutilisant le résultat de la question 24 selon lequel $\lambda_i(x, y) = \hat{\lambda}_i(\hat{x}, \hat{y})$, on obtient :

$$\int_T \lambda_k \lambda_j \, dxdy = \int_{\hat{T}} \hat{\lambda}_k \hat{\lambda}_j \,|\det(B_T)|\, d\hat{x}d\hat{y} = |\det(B_T)| \int_{\hat{T}} \hat{\lambda}_k \hat{\lambda}_j \, d\hat{x}d\hat{y}.$$

Question 27

Sur le triangle de référence $\hat{T}$, les fonctions coordonnées barycentriques sont définies par :

$$\hat{\lambda}_0(\hat{x}, \hat{y}) = 1 - \hat{x} - \hat{y}, \quad \hat{\lambda}_1(\hat{x}, \hat{y}) = \hat{x}, \quad \hat{\lambda}_2(\hat{x}, \hat{y}) = \hat{y}.$$

Il faut donc évaluer les 6 intégrales suivantes :

— Pour $k = j = 1$:

$$\int_{\hat{T}} \hat{\lambda}_1^2 \, d\hat{x}d\hat{y} = \int_0^1 \int_0^{1-\hat{x}} \hat{x}^2 \, d\hat{y}d\hat{x} = \int_0^1 \hat{x}^2(1-\hat{x}) \, d\hat{x}$$

$$= \left[\frac{\hat{x}^3}{3} - \frac{\hat{x}^4}{4}\right]_0^1 = \frac{1}{3} - \frac{1}{4} = \frac{1}{12}$$

— Pour $k = j = 2$, par symétrie :

$$\int_{\hat{T}} \hat{\lambda}_2^2 \, d\hat{x}d\hat{y} = \int_0^1 \int_0^{1-\hat{y}} \hat{y}^2 \, d\hat{x}d\hat{y} = \frac{1}{12}$$

— Pour $k = j = 0$:

$$\int_{\hat{T}} \hat{\lambda}_0^2 \, d\hat{x}d\hat{y} = \int_0^1 \int_0^{1-\hat{x}} (1-\hat{x}-\hat{y})^2 \, d\hat{y}d\hat{x} = \int_0^1 \left[-\frac{1}{3}(1-\hat{x}-\hat{y})^3\right]_0^{1-\hat{x}} \, d\hat{x}$$

$$= \int_0^1 \frac{1}{3}(1-\hat{x})^3 \, d\hat{x} = \left[-\frac{1}{12}(1-\hat{x})^4\right]_0^1 = \frac{1}{12}$$

— Pour $k = 1$, $j = 2$:

$$\int_{\hat{T}} \hat{\lambda}_1\hat{\lambda}_2 \, d\hat{x}d\hat{y} = \int_0^1 \int_0^{1-\hat{x}} \hat{x}\hat{y} \, d\hat{y}d\hat{x} = \int_0^1 \hat{x} \left[\frac{\hat{y}^2}{2}\right]_0^{1-\hat{x}} \, d\hat{x} = \frac{1}{2}\int_0^1 \hat{x}(1-\hat{x})^2 \, d\hat{x}$$

$$= \frac{1}{2}\int_0^1 (\hat{x} - 2\hat{x}^2 + \hat{x}^3) \, ddd\hat{x} = \frac{1}{2}\left(\frac{1}{2} - \frac{2}{3} + \frac{1}{4}\right) = \frac{1}{24}$$

— Pour $k = 0$, $j = 1$:

$$\int_{\hat{T}} \hat{\lambda}_0\hat{\lambda}_1 \, d\hat{x}d\hat{y} = \int_0^1 \int_0^{1-\hat{x}} (1-\hat{x}-\hat{y})\hat{x} \, d\hat{y}d\hat{x} = \int_0^1 \hat{x} \left[(1-\hat{x})\hat{y} - \frac{\hat{y}^2}{2}\right]_0^{1-\hat{x}} \, d\hat{x}$$

$$= \int_0^1 \hat{x} \left((1-\hat{x})^2 - \frac{(1-\hat{x})^2}{2}\right) \, d\hat{x} = \frac{1}{2}\int_0^1 \hat{x}(1-\hat{x})^2 \, d\hat{x} = \frac{1}{24}$$

— Pour $k = 0$, $j = 2$, par symétrie :

$$\int_{\hat{T}} \hat{\lambda}_0\hat{\lambda}_2 \, d\hat{x}d\hat{y} = \frac{1}{24}$$

Question 28

(a) Posons $I(h) = \int_{\hat{T}} h(\hat{x}, \hat{y})\, dxdy$ l'intégrale exacte et $Q(h)$ la formule de quadrature des points milieux associés suivante :

$$Q(h) = \frac{1}{6}\left(h\left(\frac{1}{2}, 0\right) + h\left(0, \frac{1}{2}\right) + h\left(\frac{1}{2}, \frac{1}{2}\right)\right).$$

Testons la méthode pour les polynômes de degrés $p + q = d$:

— Pour $d = 0$:
Alors $h(\hat{x}, \hat{y}) = 1$. On a donc $I(1) = \frac{1}{2}$.

$$Q(1) = \frac{1}{6}(1 + 1 + 1) = \frac{1}{2}.$$

La formule est donc exacte pour le degré 0.

— Pour $d = 1$:
Par symétrie, il suffit de tester $h(\hat{x}, \hat{y}) = \hat{x}$.

$$I(\hat{x}) = \int_0^1 \int_0^{1-\hat{x}} \hat{x}\, d\hat{y} d\hat{x} = \int_0^1 \hat{x}(1 - \hat{x})\, d\hat{x} = \left[\frac{\hat{x}^2}{2} - \frac{\hat{x}^3}{3}\right]_0^1 = \frac{1}{6}$$

$$Q(\hat{x}) = \frac{1}{6}\left(\frac{1}{2} + 0 + \frac{1}{2}\right) = \frac{1}{6}$$

La formule est donc exacte pour le degré 1.

— Pour $d = 2$: Il faut tester pour les monômes $\hat{x}^2$, $\hat{y}^2$ et $\hat{x}\hat{y}$.

— Pour $h(\hat{x}, \hat{y}) = \hat{x}^2$ (et $\hat{y}^2$ par symétrie) :

$$I(\hat{x}^2) = \frac{1}{12} \quad \text{(d'après la question 27).}$$

$$Q(\hat{x}^2) = \frac{1}{6}\left(\left(\frac{1}{2}\right)^2 + 0 + \left(\frac{1}{2}\right)^2\right) = \frac{1}{6} \times \frac{1}{2} = \frac{1}{12}$$

— Pour $h(\hat{x}, \hat{y}) = \hat{x}\hat{y}$:

$$I(\hat{x}\hat{y}) = \frac{1}{24} \quad \text{(d'après la question 27)}$$

$$Q(\hat{x}\hat{y}) = \frac{1}{6}\left(\frac{1}{2} \times 0 + 0 \times \frac{1}{2} + \frac{1}{2} \times \frac{1}{2}\right) = \frac{1}{6} \times \frac{1}{4} = \frac{1}{24}$$

La formule est donc également exacte pour le degré 2.

— Pour $d = 3$: On teste le monôme $h(\hat{x}, \hat{y}) = \hat{x}^3$.

$$I(\hat{x}^3) = \int_0^1 \int_0^{1-\hat{x}} \hat{x}^3 \, d\hat{y} d\hat{x} = \int_0^1 \hat{x}^3 (1 - \hat{x}) \, d\hat{x} = \left[\frac{\hat{x}^4}{4} - \frac{\hat{x}^5}{5} \right]_0^1 = \frac{1}{4} - \frac{1}{5} = \frac{1}{20}$$

$$Q(\hat{x}^3) = \frac{1}{6} \left(\left(\frac{1}{2}\right)^3 + 0 + \left(\frac{1}{2}\right)^3 \right) = \frac{1}{6} \left(\frac{1}{8} + \frac{1}{8} \right) = \frac{2}{48} = \frac{1}{24}$$

On observe que $I(\hat{x}^3) \neq Q(\hat{x}^3)$.

Finalement, la première formule de quadrature est juste pour les polynômes de degré inférieur ou égal à $d = 2$. Testons maintenant pour la formule de quadrature $R(h)$ donnée par :

$$R(h) = \frac{1}{6} \left(h\left(\frac{1}{6}, \frac{1}{6}\right) + h\left(\frac{1}{6}, \frac{2}{3}\right) + h\left(\frac{2}{3}, \frac{1}{6}\right) \right)$$

— Pour $d = 0$:
Alors $h(\hat{x}, \hat{y}) = 1$. Comme précédemment, on a $I(1) = \frac{1}{2}$.

$$R(1) = \frac{1}{6} (1 + 1 + 1) = \frac{1}{2}.$$

La formule est donc exacte pour le degré 0.

— Pour $d = 1$:
Par symétrie, il suffit de tester $h(\hat{x}, \hat{y}) = \hat{x}$. On a déjà calculé $I(\hat{x}) = \frac{1}{6}$.

$$R(\hat{x}) = \frac{1}{6} \left(\frac{1}{6} + \frac{1}{6} + \frac{2}{3} \right) = \frac{1}{6} \left(\frac{2}{6} + \frac{4}{6} \right) = \frac{1}{6} \times 1 = \frac{1}{6}$$

La formule est donc exacte pour le degré 1.

— Pour $d = 2$: Il faut tester pour les monômes $\hat{x}^2$, $\hat{y}^2$ et $\hat{x}\hat{y}$.

— Pour $h(\hat{x}, \hat{y}) = \hat{x}^2$ (et $\hat{y}^2$ par symétrie) :

$$\begin{aligned} R(\hat{x}^2) &= \frac{1}{6} \left(\left(\frac{1}{6}\right)^2 + \left(\frac{1}{6}\right)^2 + \left(\frac{2}{3}\right)^2 \right) \\ &= \frac{1}{6} \left(\frac{1}{36} + \frac{1}{36} + \frac{16}{36} \right) = \frac{1}{6} \times \frac{18}{36} = \frac{1}{12} \end{aligned}$$

— Pour $h(\hat{x}, \hat{y}) = \hat{x}\hat{y}$:

$$\begin{aligned} R(\hat{x}\hat{y}) &= \frac{1}{6} \left(\frac{1}{6} \times \frac{1}{6} + \frac{1}{6} \times \frac{2}{3} + \frac{2}{3} \times \frac{1}{6} \right) \\ &= \frac{1}{6} \left(\frac{1}{36} + \frac{4}{36} + \frac{4}{36} \right) = \frac{1}{6} \times \frac{9}{36} = \frac{1}{24} \end{aligned}$$

La formule est donc également exacte pour le degré 2.

— Pour $d = 3$: On teste le monôme $h(\hat{x}, \hat{y}) = \hat{x}^3$. On sait d'après le calcul précédent que $I(\hat{x}^3) = \frac{1}{20}$.

$$\begin{aligned} R(\hat{x}^3) &= \frac{1}{6}\left(\left(\frac{1}{6}\right)^3 + \left(\frac{1}{6}\right)^3 + \left(\frac{2}{3}\right)^3\right) \\ &= \frac{1}{6}\left(\frac{1}{216} + \frac{1}{216} + \frac{64}{216}\right) = \frac{1}{6} \times \frac{66}{216} = \frac{11}{216} \end{aligned}$$

On observe que $I(\hat{x}^3) \neq R(\hat{x}^3)$.

Finalement, la deuxième formule de quadrature est également exacte pour les polynômes de degré inférieur ou égal à $d = 2$.

3. Le problème discret

Remarque.
Les fonctions des questions 28(i), 28(j), 29(l) et 29(m) sont incluses dans le code fourni dans l'annexe B.

Question 28 (suite)

(d) On note I_h l'application de V_h dans $\mathbb{R}^G$ définie par

$$\forall\, v \in V_h, \quad I_h(v) = (v(M_k))_{0 \leq k \leq G-1}.$$

Soit $v \in \ker I_h$. On a pour tout $0 \leq k \leq G-1$, $v(M_k) = 0$. Soit $l \in [\![0, k-1]\!]$ et soit T_l un triangle du maillage, de sommets A_0, A_1 et A_2.

$v_{|T_l}$ est affine et d'après la question 17, une application affine est entièrement déterminée par ses valeurs aux trois sommets.

Or A_0, A_1, A_2 sont des noeuds. Donc $v(A_i) = 0$ pour tous $i \in \{0, 1, 2\}$ et ainsi, $v_{|T_l} = 0$. Ceci étant vrai pour tout triangle, $v = 0$ sur $\overline{\Omega}$, et on en déduit que I_h est injective.

Montrons maintenant que I_h est surjetive. Soit $(y_0, y_1, \ldots, y_{G-1}) \in \mathbb{R}^G$. On a :

$$v_{|T_l}(x, y) = \sum_{j=0}^{2} y_{l_j} \lambda_j^{T_l}(x, y), \quad \text{où les } y_{l_j} \text{ sont les images par } v \text{ des } y_l \text{ aux noeuds.}$$

On veut vérifier que v est continue sur $\overline{\Omega}$. v étant affine sur chaque triangle, on veut vérifier sa continuité sur les arrêtes des triangles.

Soient T_1, T_2 deux triangles qui ont une arrête $A = [A_0, A_1]$ commune. Alors $\left(v_{|T_i}\right)_{|A}$ est affine en une variable (puisque l'on est sur un segment). De plus, $\left(v_{|T_1}\right)_{|A}(A_i) = \left(v_{|T_2}\right)_{|A}(A_i)$, donc $v_{|T_1} = v_{|T_2}$. Par suite, v est continue sur $\overline{\Omega}$ et donc v est bien un élément de V_h.

Comme I_h est un isomorphisme, alors $\dim \mathbb{R}^G = \dim V_h = G$.

(e) w_k est l'image réciproque de e_k par I_h où e_k est le k-ième vecteur de la base canonique de $\mathbb{R}^G$. Comme d'après la question précédente, I_h est un isomorphisme,

et que l'image réciproque d'une base par un isomorphisme est une base, alors les w_k forment une base de V_h.

(f)

1. Supposons que M_k n'est pas un sommet de T. Soient M_1, M_2, M_3 les trois sommets de T. Ainsi, $k \notin \{1, 2, 3\}$. Par définition, on a $w_k(M_l) = \delta_{k,l}$. Mais M_k n'appartenant pas au triangle, w_k s'annule aux trois sommets. Comme de plus $(w_k)_{|T}$ est affine, on déduit de la question 17 qu'elle est identiquement nulle. Par suite, $(w_k)_{|T} = 0$.
2. Supposons maintenant que $M_k \in T$. Soient A_0, A_1, A_2 les sommets de T et $\lambda_0, \lambda_1, \lambda_2$ les coordonnées barycentriques associées. Alors il existe $i \in \{0, 1, 2\}$ tel que $M_k = A_i$. Comme $(w_k)_{|T}$ est affine, que $(w_k)_{|T}(A_i) = 1$ et que $(w_k)_{|T}(A_m) = 0$ pour $m \neq i$, alors $(w_k)_{|T}$ est exactement $(w_k)_{|T} = \lambda_i$.
3. D'après la question précédente, on a que $(w_k)_{|T}$ et $(w_j)_{|T}$ sont des coordonnées barycentriques. Comme de plus $F_T^{-1}(M_k) = \hat{A}_l$ et $F_T^{-1}(M_j) = \hat{A}_s$, alors on a $(w_k)_{|T} = \lambda_l$ et $(w_j)_{|T} = \lambda_s$. Ainsi, d'après la question 25, on a :

$$\nabla w_k = (B_T^{-1})^T \nabla \hat{\lambda}_l \quad \text{et} \quad \nabla w_j = (B_T^{-1})^T \nabla \hat{\lambda}_s.$$

Par suite,

$$\nabla w_k \cdot \nabla w_j = \left((B_T^{-1})^T \nabla \hat{\lambda}_l\right)^T \left((B_T^{-1})^T \nabla \hat{\lambda}_s\right).$$

En développant, on obtient :

$$\nabla w_k \cdot \nabla w_j = \left(\nabla \hat{\lambda}_l\right)^T B_T^{-1} (B_T^{-1})^T \nabla \hat{\lambda}_s.$$

(g) Soit $u^h \in V_h$. D'après la question 28(e), les w_k forment une base de V_h. On peut donc écrire u^h sous la forme :

$$u^h = \sum_{j=0}^{G-1} u_j w_j, \quad u_j \in \mathbb{R}.$$

On en déduit par bilinéarité de a que

$$\begin{aligned} a\left(u^h, v_h\right) &= a\left(\sum_{j=0}^{G-1} u_j w_j, v_h\right) \\ &= \sum_{j=0}^{G-1} u_j a(w_j, v_h), \qquad \forall\, v_h \in V_h. \end{aligned}$$

Comme a est linéaire, il suffit de vérifier pour tous les vecteurs de la base. Ainsi, en posant $l(w_k) = b_k$ et en utilisant le fait que $a(w_j, w_k) = a_{k,j}$,

$$\sum_{j=0}^{G-1} u_j a(w_j, w_k) = l(w_k)$$

devient

$$\sum_{j=0}^{G-1} u_j a_{k,j} = b_k,$$

ce qui correspond exactement à la k-ième ligne du système $AU = B$ où U est le vecteur colonne contenant les u_j.

(h) Soit $V = (v_0, \ldots, v_{G-1}) \in \mathbb{R}^G$. On pose $v = \sum_{j=0}^{G-1} v_j w_j$. Alors $v = 0$ si et seulement si $V = 0$. De plus,

$$V^T A V = \sum_{k=0}^{G-1} \sum_{j=0}^{G-1} v_k a_{k,j} v_j = \sum_{k=0}^{G-1} \sum_{j=0}^{G-1} v_k a(w_j, w_k) v_j$$

$$= a\left(\sum_{j=0}^{G-1} v_j w_j, \sum_{k=0}^{G-1} v_k w_k\right) = a(v, v).$$

Comme

$$a(v, v) = \int_\Omega (\eta \, ||\nabla v\,||^2 + \lambda v^2)\, dxdy \geq 0,$$

que $\eta(x, y) \geq \eta_0 > 0$ et $\lambda > 0$, alors par positivité de l'intégrale,

$$a(v, v) = 0 \quad \Longleftrightarrow \quad v = 0 \quad \Longleftrightarrow \quad V = 0.$$

On en déduit que A est définie positive et donc que A est inversible.

Question 29

(k) Soient $V = (v_0, \ldots, v_{G-1})^T \in \mathcal{M}_{G,1}\left(\mathbb{R}^G\right)$ et $v = \sum_{k=0}^{G-1} v_k w_k \in V_h$. Alors, comme le montre le calcul intermédiaire de la question 28(h),

$$V^T A V = a(v, v).$$

De plus,

$$a(v,v) = \int_\Omega (\eta \, ||\nabla v \, ||^2 + \lambda v^2) \, dxdy = \int_\Omega \eta \, ||\nabla v \, ||^2 \, dxdy + \lambda \int_\Omega v^2 \, dxdy$$

$$= \int_\Omega \eta \, ||\nabla v \, ||^2 \, dxdy + \lambda \, ||v||^2_{L^2(\Omega)}.$$

Comme $\eta(x,y) > 0$ pour tout $(x,y) \in \Omega$, à (x,y) fixé, η est un scalaire. On a donc $\eta \, ||\nabla v \, ||^2 = ||\sqrt{\eta} \nabla v \, ||^2$. Finalement,

$$a(v,v) = \int_\Omega ||\sqrt{\eta} \nabla v \, ||^2 \, dxdy + \lambda \, ||v||^2_{L^2(\Omega)}.$$

$$= ||\sqrt{\eta} \nabla v \, ||^2_{L^2(\Omega)^2} + \lambda \, ||v||^2_{L^2(\Omega)}.$$

Remarque.
Les réponses aux questions 30 à 32(d) sont données dans le chapitre **??**sec :test-num]Tests numériques.

Chapitre II.
Tests Numériques

Remarque.
La fonction de la question 32(b) est incluse dans le code fourni dans l'annexe B. Le reste du code lié à cette partie est intégré au main.

Question 30

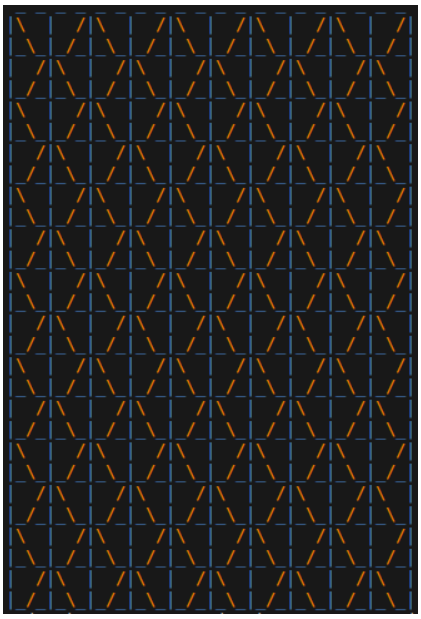


FIGURE II.1 – Affichage du maillage $N \times M = 10 \times 14$ dans le terminal

Question 31

Soient $h_1 = \frac{2a}{N}, h_2 = \frac{2b}{M}$ et $h = \max(h_1, h_2)$. Soit $(x, y) \in \Omega$ quelconque. Alors en particulier, $x \in]-a, a[$ et $y \in]-b, b[$. Ainsi,

$$\begin{aligned} x \in]-a, a[\quad &\iff \quad -a < x < a \quad \iff \quad 0 < x + a < 2a \\ &\iff \quad 0 < \frac{x+a}{h_1} < N \quad \iff \quad 0 < i < N. \end{aligned}$$

D'où $i \leq N - 1$. De la même manière,

$$\begin{aligned} y \in]-b, b[\quad &\iff \quad -b < y < b \quad \iff \quad 0 < y + b < 2b \\ &\iff \quad 0 < \frac{y+b}{h_2} < M \quad \iff \quad 0 < j < M. \end{aligned}$$

Ce qui donne $j \leq M - 1$.

Par définie de la partie entière,

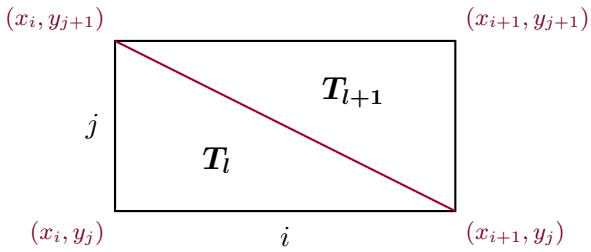
$$\begin{aligned} i = \mathrm{E}\left[\frac{x+a}{h_1}\right] &\Longrightarrow i \leq \frac{x+a}{h_1} < i + 1 \\ &\iff \quad ih_1 \leq x + a \leq (i+1)h_1 \\ &\iff \quad -a + ih_1 \leq x < -a + (i+1)h_1 \\ &\iff \quad x_i \leq x < x_{i+1}. \end{aligned}$$

De manière analogue,

$$\begin{aligned} j = \mathrm{E}\left[\frac{y+b}{h_2}\right] &\Longrightarrow j \leq \frac{y+b}{h_2} < j + 1 \\ &\iff \quad jh_2 \leq y + b \leq (j+1)h_2 \\ &\iff \quad -b + jh_2 \leq y < -b + (j+1)h_2 \\ &\iff \quad y_j \leq y < y_{j+1}. \end{aligned}$$

On en déduit donc que $(x, y) \in [x_i, x_{i+1}[\times [y_j, y_{j+1}[$. Or $[x_i, x_{i+1}[\times [y_j, y_{j+1}[\subset T_l \cup T_{l+1}$. En effet, d'après la numérotation des noeuds que l'on a réalisé dans

la partie 2, les sommets de T_l et T_{l+1} avec $l = (N+1)j + i$ sont respectivement $\{ (x_i, y_j), (x_i, y_{j+1}), (x_{i+1}, y_j) \}$ et $\{ (x_i, y_{j+1}), (x_{i+1}, y_{j+1}), (x_{i+1}, y_j) \}$ (ou bien $\{ (x_i, y_j), (x_i, y_{j+1}), (x_{i+1}, y_{j+1}) \}$ et $\{ (x_i, y_j), (x_{i+1}, y_{j+1}), (x_{i+1}, y_j) \}$ selon le sens de la partition du rectangle) :



On a donc $(x, y) \in T_l \cup T_{l+1}$ et donc $(x, y) \in T_l$ ou $(x, y) \in T_{l+1}$.

Question 32

(a) D'après la question 3,

$$u^p(x, y) = \frac{\cos(m\pi x)\cos(m\pi y)}{2m^2\pi^2 + 1}$$

Pour simplifier l'écriture, on pose $C = \frac{1}{2m^2\pi^2+1}$. On a d'une part

$$\frac{\partial u^p}{\partial x}(x, y) = -Cm\pi \sin(m\pi x)\cos(m\pi y),$$

et d'autre part

$$\frac{\partial u^p}{\partial y}(x, y) = -Cm\pi \cos(m\pi x)\sin(m\pi y).$$

On calcule ensuite les dérivées secondes :

$$\frac{\partial^2 u^p}{\partial x^2}(x, y) = -Cm^2\pi^2 \cos(m\pi x)\cos(m\pi y) = -m^2\pi^2 u^p(x, y)$$

et

$$\frac{\partial^2 u^p}{\partial y^2}(x, y) = -m^2\pi^2 u^p(x, y).$$

On pose $\eta(x, y) = 1 - \frac{\eta_0}{4}(x^2 + y^2)$. Ainsi,

$$\frac{\partial}{\partial x}\left(\eta \frac{\partial u^p}{\partial x}\right)(x, y) = \frac{\eta_0 x}{2} Cm\pi \sin(m\pi x)\cos(m\pi y) - m\check{s}\pi^2 u^p(x, y)\eta(x, y) := A(x, y)$$

et

$$\frac{\partial}{\partial y}\left(\eta\frac{\partial u^p}{\partial y}\right)(x,y) = \frac{\eta_0 x}{2}Cm\pi\cos(m\pi x)\sin(m\pi y) - m\check{s}\pi^2 u^p(x,y)\eta\,(x,y) := B(x,y).$$

En reprenant l'équation (1) du sujet, avec $\lambda = 1$, on obtient alors

$$-\frac{\partial}{\partial x}\left(\eta\frac{\partial u^p}{\partial x}\right)(x,y) - \frac{\partial}{\partial y}\left(\eta\frac{\partial u^p}{\partial y}\right)(x,y) + \lambda u^p(x,y) = f(x,y)$$

$$\iff \quad f(x,y) = -A(x,y) - B(x,y) + u^p(x,y)$$

En remplaçant $A(x,y)$ et $B(x,y)$ par leure expression donnée plus haut puis en factorisant, on obtient finalement :

$$f(x,y) = -\frac{\eta_0 m\pi}{4m^2\pi^2+2}(x\sin(m\pi x)\cos(m\pi y) + y\cos(m\pi x)\sin(m\pi y)) \\ + (2m^2\pi^2\eta\,(x,y)+1)u^p(x,y).$$

(c) En testant pour $m = 1.$ et $N = M = 2/h \in \{\,4, 8, 16, 20, 32, 64, 128\,\}$, nous avons obtenu les erreurs relatives suivantes :

```
N e0 e1 e2
4 0.00137758 0.00137758 0.00137758
8 0.0425056 0.132984 0.0691583
16 0.00954852 0.0656942 0.022318
20 0.00602898 0.052484 0.0146806
32 0.00232045 0.032755 0.00590366
64 0.000575973 0.0163662 0.00149802
128 0.000143869 0.0081817 0.000378497
```

Avec le passage au log et un test sur un plus grand nombre de valeurs de N, cela nous a permis d'obtenir le graphe suivant :

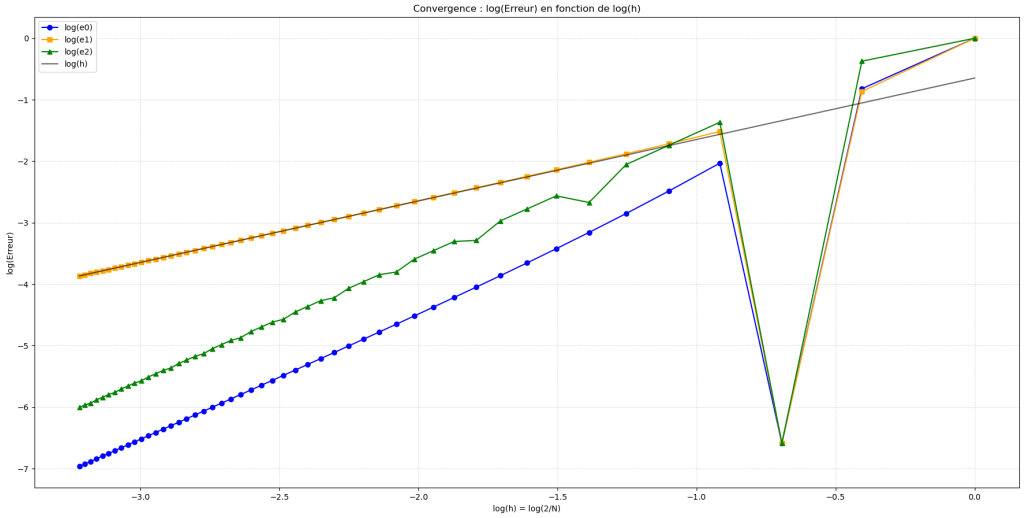


FIGURE II.2 – Tracé des courbes d'erreurs

On remarque que pour $N = 4$, l'erreur plonge, ce qui indique que la solution approchée correspond presque exactement à la solution réelle : les noeuds du maillage semblent donc coïnder avec les points de la solution exacte.

(d) A partir de la question 31, nous avons pu créer une fonction eval_uh que nous avons évalué en 50 points pour différentes valeurs de m. En comparant avec la fonction log(h), nous avons obtenu les graphes suivants :

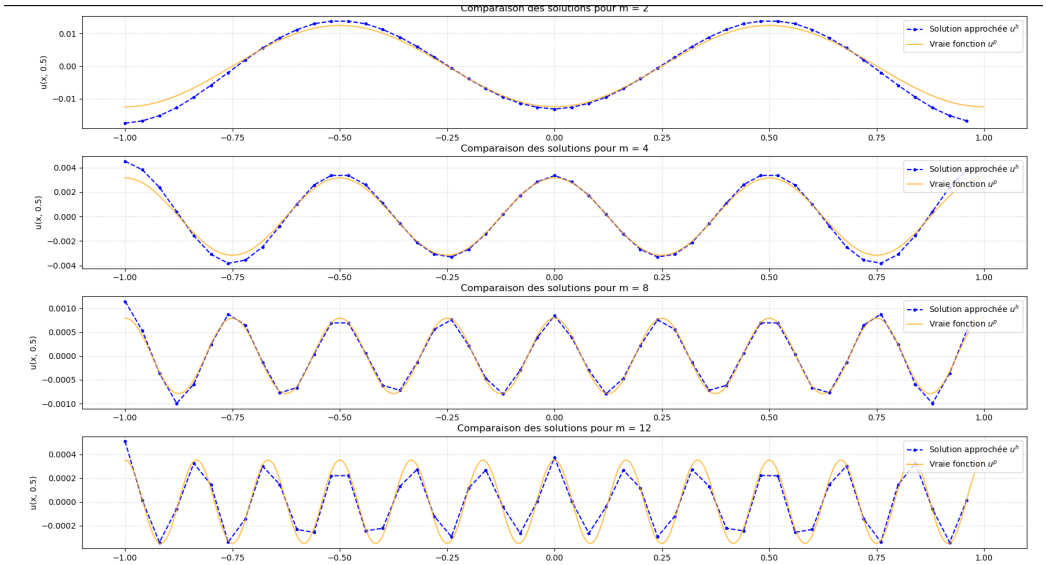


FIGURE II.3 – Tracé des courbes pour $x \in [-1, 1]$

On observe que pour des petites valeurs de m $(2, 4)$ le maillage $N = 32$ est suffisant, mais lorsque m grandit, le maillage n'est plus assez fin pour capter les variations de la fonction, et donc l'erreur est bien plus grande.

Chapitre III.
Structure du programme

La structure du code étant un élément essentiel dans un projet, nous y avons porté un soin tout particulier. Il est structuré en trois fichiers principaux : fonctions.cpp, main.cpp et tests.cpp ; contenant respectivement les fonctions relatives au projet, les tests numériques et des tests pour la plupart des fonctions codées.

Afin de structurer davantage notre code, nous avons opté pour l'utilisation de classes. Ce choix à engendré bien des questionnements notamment sur la manière de définir nos classes et la pertinence réelle de leur utilisation dans ce projet. Nous n'avons au final, eu l'utilité que d'une seule, la classe triangle.cpp. Elle nous a permis, entre autres, de plus agréablement manipuler les maillages, qui n'est représenté maintenant que par un tableau de Triangle au lieu d'une matrice. Cette classe est défini par ses 3 attributs : $n1, n2, n3$ qui correspondent au trois sommets, avec la numérotation des noeuds dans le maillage. La surcharge de l'opérateur == permet une comparaison plus pratique des triangles (cette surcharge est presque obligatoire pour faire les tests dans de bonnes conditions).

Nous avons également fourni un fichier tests.cpp, comme annoncé au début du paragraphe, pour s'assurer que les fonctions codées tout au long du projet, soient correctes et donc que nous travaillons sur de bonnes bases.

Toutes nos tentatives ne sont pas utilisées dans la version finale. La première aura été de créer une classe noeud.cpp, que nous avons enlevé faute d'utilité et cela s'explique par la très bonne numérotation des sommets dans le maillage qui est autosuffisante. La deuxième est la classe elementvh.cpp qui est visible dans le code en annexe. Encore une fois, la manière proposée d'aborder le problème nous a paru plus appropriée et moins lourde que l'utilisation d'une classe. Dans ce que nous visualisions, nous l'utilisions que dans une seule fonction (cf. **??**), ce qui justifie son retrait.

Chapitre IV.
Annexe A : main

L'annexe ci-dessous contient la fichier principal du programme. En particulier, nous y avons inclut les fonctions spécifiques aux tests numériques (affichage du maillage, évaluation de u^h en un point de Ω, ...) et à la génération des .txt à partir desquels nous avons pu visualiser nos résultats (questions 30, 32(c) et 32(d)).

main.cpp

```
#include <iostream>
#include <cmath>
#include <fstream>

#include <functional>

#include "fonctions.hpp"

using namespace std;

const string BLEU  = "\033[34m";
const string ORANGE = "\033[38;5;208m";
const string BLANC = "\033[0m";

void affiche_maillage(int N, int M){

    cout << "Affichage du maillage pour N = "<< N << ", M = "<<M<<".\
    n";

    // Affichage de la première ligne
    string buffer_haut ="" + BLEU;
    for (int c = 0; c < N; c++) buffer_haut += " _ _";

    buffer_haut += BLANC + "\n";
    cout << buffer_haut ;

    for (int l = M-1; l >= 0; l--){

        string buffer_mil = "";
```

```
        string buffer_bas = "";

        for (int c = 0; c < N; c++){

            bool direction = ((l + c) % 2 == 0);

            if (c == 0){
                buffer_mil += BLEU + "|" + BLANC;
                buffer_bas += BLEU + "|" + BLANC;
            }

            if (direction){ // CAS 1 : direction "croissante"
                buffer_mil += ORANGE + "  /" + BLEU + "|" + BLANC;
                buffer_bas += BLEU + "_"+ ORANGE +"/" + BLEU + "_|" + BLANC;
            } else {  // CAS 2 : direction "décroissante"
                buffer_mil += ORANGE + "\\  "+ BLEU + "|"+ BLANC ;
                buffer_bas += BLEU + "_"+ ORANGE +"\\"+BLEU + "_|" + BLANC;
            }
        }

        cout << buffer_mil << endl << buffer_bas << endl;
    }

}


double up(double x, double y, int m){
    double c = (2.*m*m*M_PI*M_PI + 1);
    return (cos(m*M_PI*x)*cos(m*M_PI*y)) /c ;
}

// Se référer à la question 32)a. Pour la formule
double f(double x, double y, int m, double eta0, double (* eta) (double,double)){
    double k = -(eta0*m*M_PI)/(4.*m*m*M_PI*M_PI +2);
    double t = x*sin(m*M_PI*x)*cos(m*M_PI*y) + y*cos(m*M_PI*x)*sin(m*M_PI*y);
    double o = 2*m*m*M_PI*M_PI*eta(x,y) +1;
    return k*t + o*up(x,y,m);
}

// Cas eta0=0
double f0(double x, double y, int m){
    return f(x,y,m,0,[](double,double){return 1.;});
}

// Cas eta0=1
double f1(double x, double y, int m){
    return f(x,y,m,1.,[](double,double){return 1.;});
}
```

```
double eta(double eta0, double x, double y){
    return 1. - eta0/4 * (x*x+y*y);
}

double eval_uh(double x, double y, vector<double> uh, maillage TRG, int N, int M, double a, double b){
    // On cherche à identifier dans quel triangle se trouve (x,y)
    double h1 = (2*a)/N;
    double h2 = (2*b)/M;
    int i = (int) ((x+a)/h1);
    int j = (int) ((y+b)/h2);

    int l = 2*j*N+2*i;

    // (x,y) est dans T_l ou T_{l+1} (q31)
    // On a besoin du signe des lambdas

    for (int t = l; t < l + 2; t++){
        Triangle T = TRG[t];

        duplix xs_ys = CoordsTrig(a, b, N, M, T);
        vector<double> xs = get<0>(xs_ys);
        vector<double> ys = get<1>(xs_ys);

        matrix BT = CalcMatBT(xs, ys);

        double detBT = BT[0][0]*BT[1][1] - BT[0][1]*BT[1][0];

        // Inversion BT
        matrix inv_BT = {{BT[1][1]/detBT, -BT[0][1]/detBT},
                         {-BT[1][0]/detBT, BT[0][0]/detBT}};

        // Calcul des lambdas
        double lambda_1 = inv_BT[0][0]*(x - xs[0]) + inv_BT[0][1]*(y - ys[0]);
        double lambda_2 = inv_BT[1][0]*(x - xs[0]) + inv_BT[1][1]*(y - ys[0]);
        double lambda_0 = 1. - lambda_1 - lambda_2;

        // Test signe
        if (lambda_0 >= 0 && lambda_1 >= 0 && lambda_2 >= 0){

            // Alors (x,y) est dans ce triangle
            double eval = lambda_0*uh[T.get(0)] + lambda_1*uh[T.get(1)] + lambda_2*uh[T.get(2)];

            return eval;
        }
    }

    return 0.;
```

```
}

int main(){
    cout << "Début des tests numériques\n";

    // Déclaration des paramètres
    const double a = 1.;
    const double b = 1.;
    int m = 1;

    auto lambda = [](double,double){ return 1.;};

    auto eta1 = lambda;

    // Tests numériques

    affiche_maillage(10,14);

    vector<int> valeurs_tests = {4,8,16,20,32,64};
    vector<int> valeurs_tests1(49);
    for (int k = 0; k < 49;k++) valeurs_tests1[k] = k+2;

    // Création d'un fichier, pour stocker les erreurs pour l'affichage

    ofstream fichier("erreurs.txt");
    ofstream fichier2("log_erreurs.txt");
    if (!fichier.is_open() || !fichier2.is_open()) {
        cout << "Erreur lors de l'ouverture d'un fichier.\n";
        return 1;
    }

    fichier << "N e0 e1 e_2\n";
    fichier2 << "N log(e0) log(e1) log(e_2)\n";

    // Calculs pour la question 32(c)
    cout << "Calcul des erreurs en cours...\n";

    for (int N : valeurs_tests1){
        maillage maillageNxN = maillageTR(N,N);

        vector<double> B = scdmembre(f0,N,N,maillageNxN,a,b,m);

        vector<double> uh = bicg_stab(B,maillageNxN,N,N,a,b,1e-10,100000,eta1);

        auto up0 = [](double x, double y, int m){ return up(x,y,m);};
        vector<double> tab_err = erreurs(up0,uh,maillageNxN,N,N,a,b,m);
        vector<double> tab_log_err(tab_err.size());

        for (int i = 0; i < tab_log_err.size(); i++){
```

```
            tab_log_err[i] = log(tab_err[i]);
        }

        cout << "log(e0(2/" << N << ")) = " << tab_log_err[0] << ", ";
        cout << "log(e1(2/" << N << ")) = " << tab_log_err[1] << ", ";
        cout << "log(e2(2/" << N << ")) = " << tab_log_err[2] << ", ";
        cout << "log(2/" << N << ") = " << log(2./N) << endl;

        cout << "e0(2/" << N << ") = " << tab_err[0] << ", ";
        cout << "e1(2/" << N << ") = " << tab_err[1] << ", ";
        cout << "e2(2/" << N << ") = " << tab_err[2] << " \n";

        fichier << N << " " << tab_err[0] << " " << tab_err[1] << " " << tab_err[2] << "\n";
        fichier2 << N << " " << tab_log_err[0] << " " << tab_log_err[1] << " " << tab_log_err[2] << "\n";
    }

    // Début de tests question 32(d), maillage 32x32, 50 points par courbe

    cout << "Génération des courbes en cours...\n";

    ofstream fichier1("courbes.txt");
    if (!fichier1.is_open()) {
        cout << "Erreur lors de l'ouverture du fichier.\n";
        return 1;
    }

    fichier1 << "m x u^h u^p\n";
    double pas = 2./50;
    int N_courbes = 32;

    maillage maillage_courbes = maillageTR(N_courbes, N_courbes);

    // Tests sur differentes valeurs de m
    vector<int> valeurs_m = {2, 4, 8, 12};

    for (int m_courbe : valeurs_m){
        auto f1_courbe = [](double x, double y, int m_courbe){ return f1(x, y, m_courbe);};
        vector<double> B_courbe = scdmembre(f1_courbe, N_courbes, N_courbes, maillage_courbes, a, b, m_courbe);

        auto eta1_courbe = [](double x, double y){return eta(1.,x,y);};
        vector<double> uh_courbe = bicg_stab(B_courbe, maillage_courbes, N_courbes, N_courbes, a, b, 1e-10, 100000, eta1_courbe);
```

```

        double ctr = -1.;

        while (ctr <= 1.){
            double u_app = eval_uh(ctr, 0.5, uh_courbe, maillage_courbes, N_courbes, N_courbes, a, b);
            double u_exa = up(ctr, 0.5, m_courbe);

            fichier1 << m_courbe << " " << ctr << " " << u_app << " " << u_exa << "\n";
            ctr += pas;
        }

    }

    cout << "Tests terminés, les fichiers .txt sont prêts !\n";

}
```

Chapitre V.
Annexe B : fonctions

L'annexe ci-dessous contient toutes les fonctions demandées dans les questions qui n'apparaissent pas explicitement dans le chapitre "Réponses aux questions théoriques" (cf. remarques en début de chaque partie du chapitre sus-mentionné). Nous avons pris la liberté d'ajouter quelques fonctions supplémentaires pour en alléger d'autres.

fonctions.hpp

```
#ifndef FONCTIONS_H
#define FONCTIONS_H

#include <tuple>
#include <vector>


#include "triangle.hpp"

using namespace std;

typedef vector<vector<double>> matrix;
typedef tuple<vector<double>,vector<double>> duplix;
typedef vector<Triangle> maillage;

vector<double> Subdiv(double a, int N);

int numgb(int N, int M, int i, int j);

tuple<int,int> invnumgb(int N, int M,int s);

maillage maillageTR(int N, int M);

duplix CoordsTrig(double a, double b, int N,int M, Triangle T);

matrix CalcMatBT(vector<double> xs, vector<double> ys);

vector<double> integ_eta_triang(double (* eta)(double,double),
    maillage TRG,int N, int M,double a,double b);
```

```

matrix DiffTerm(duplix xs_ys, double val);

matrix ReacTerm(duplix xs_ys);

vector<double> matvec(vector<double> V, maillage TRG, int N, int M,
    double a, double b, double (* eta)(double,double));

#endif

vector<double> scdmembre(double rhsf(double,double, int), int N, int
    M, maillage TRG, double a, double b, int m);

double normL2(vector<double> V,maillage TRG, int N, int M, double a,
    double b);

double normL2Grad(vector<double> V, maillage TRG, int N, int M,
    double a, double b);

double pdt_sc(vector<double> u, vector<double> v);

vector<double> cl_vec(double lbd, vector<double> u, double mu, vector
    <double> v);

double max_abs(vector<double> V);

vector<double> bicg_stab(vector<double> B,maillage TRG, int N, int M,
     double a,
    double b, double tol,int max_it, double (* eta) (double,double));

vector<double> erreurs(double (* solExa) (double,double, int),vector<
    double> uh,maillage TRG, int N, int M, double a, double b, int m)
    ;
```

fonctions.cpp

```
#include <stdlib.h>
#include <cmath>

#include "fonctions.hpp"



// Fonction qui renvoie une subdivision uniforme de [-a,a]
vector<double> Subdiv(double a, int N){
    vector<double> subdiv = vector<double>(N+1);
    double pas = 2*a / N;

    for (int k = 0; k <= N; k++) subdiv[k] = -a + k*pas;

    return subdiv;
}
```

```

// Fonction qui renvoie le numéro associé à des coordonnées
int numgb(int N, int M, int i, int j){ return (N+1)*j + i; }

//Fonction qui étant donné entier s valide, donne les coordonnées du noeud
tuple<int,int> invnumgb(int N, int M,int s){
    return {s % (N+1),s / (N+1)};
}

//Fonction qui étant donné un pavé de NM rectangles renvoie un TRG
//triangulaire sous forme d'une liste qui contient les triangles
maillage maillageTR(int N, int M){
    maillage TRG(2*N*M);
    int k = 0;
    bool direction = true; // true pour "/" et false pour "\"

    for (int l = 0; l < M; l++){
        for (int c = 0; c < N ; c++){

            //Pour alterner le sens des triangles à chaque petit rectangle
            direction = (l+c)%2 == 0;

            // Calcul des coordonnées des quatres sommets du rectangle
            int bas_g = numgb(N,M,c,l);
            int bas_d = numgb(N,M,c+1,l);
            int haut_g = numgb(N,M,c,l+1);
            int haut_d = numgb(N,M,c+1,l+1);

            // Création des triangles
            if (direction){
                TRG[k] = Triangle(bas_g,haut_g,haut_d);
                TRG[k+1] = Triangle(bas_g,bas_d,haut_d);
            } else {
                TRG[k] = Triangle(bas_g,haut_g,bas_d);
                TRG[k+1] = Triangle(haut_g,haut_d,bas_d);
            }

            k += 2;
        }
    }

    return TRG;
}


// Fonction qui renvoie les coordonnées des points du triangle
duplix CoordsTrig(double a, double b, int N,
    int M, Triangle T){
    vector<double> xs = vector<double>(3);
```

```
    vector<double> ys = vector<double>(3);

    for (int k = 0; k < 3; k++){
        tuple<int,int> tup = invnumgb(N,M,T.get(k));
        xs[k] = -a + (2*a)/N * get<0>(tup);
        ys[k] = -b + (2*b)/M * get<1>(tup);

    }

    return {xs,ys};
}


// Fonction qui renvoie la jacobienne de F_T
// Voir question 20, elle justifie ce calcul
matrix CalcMatBT(vector<double> xs, vector<double> ys){
    double bt_00 = xs[1] - xs[0];
    double bt_10 = ys[1] - ys[0];
    double bt_01 = xs[2] - xs[0];
    double bt_11 = ys[2] - ys[0];

    return {{bt_00,bt_01},{bt_10,bt_11}};
}




double integ_triangle(double (* f)(double,double), Triangle T,int N,
    int M,double a,double b){

    // Calcul du determinant de B_T
    duplix xs_ys = CoordsTrig(a,b,N,M,T);
    vector<double> xs = get<0>(xs_ys);
    vector<double> ys = get<1>(xs_ys);

    double det = abs((xs[1]-xs[0])*(ys[2]-ys[0]) - (ys[1]-ys[0])*(xs
    [2]-xs[0]));

    // On determine les milieux des côtes de T
    double m0x = (xs[0] + xs[1])/2.;
    double m0y = (ys[0] + ys[1])/2.;
    double m1x = (xs[1] + xs[2])/2.;
    double m1y = (ys[1] + ys[2])/2.;
    double m2x = (xs[0] + xs[2])/2.;
    double m2y = (ys[0] + ys[2])/2.;

    // Calculs de f \circ F_T en les milieux des côtés du triangles T
    ^
    double f0 = f(m0x,m0y);
    double f1 = f(m1x,m1y);
    double f2 = f(m2x,m2y);

```

```
    return 1./6. * det * (f0 + f1 + f2);
}


vector<double> integ_eta_triang(double (* eta)(double,double),
    maillage TRG,int N, int M,double a,double b){

    vector<double> ET = vector<double>(TRG.size());
    int k =0;

    // Parcours du maillage
    for (Triangle T : TRG){


        ET[k] = integ_triangle(eta,T,N,M,a,b);
        k++;
    }

    return ET;
}


matrix DiffTerm(duplix xs_ys, double val){
    vector<double> xs = get<0>(xs_ys);
    vector<double> ys = get<1>(xs_ys);

    matrix BT = CalcMatBT(xs,ys);
    double det = (xs[1]-xs[0])*(ys[2]-ys[0]) - (ys[1]-ys[0])*(xs[2]-xs[0]);

    //Construction de la transposée de l'inverse de BT
    matrix trsp_inv_BT = {{0,0},{0,0}};
    trsp_inv_BT[0][0] = 1/det * BT[1][1];
    trsp_inv_BT[0][1] = -1/det * BT[1][0];
    trsp_inv_BT[1][0] = -1/det * BT[0][1];
    trsp_inv_BT[1][1] = 1/det * BT[0][0];


    //Calcul des Nabla_lambda^
    vector<double> NLcha0 = {{-1,-1}}; //lambdacha0 = (1-x^-y^)
    vector<double> NLcha1 = {{1,0}}; //lambdacha1 = x^
    vector<double> NLcha2 = {{0,1}}; //lambdacha2 = y^
    //Calcul des Nabla_lambda = BT*Nabla_lambdacha
    double nl00 = trsp_inv_BT[0][0] * NLcha0[0] + trsp_inv_BT[0][1] * NLcha0[1];
    double nl01 = trsp_inv_BT[1][0] * NLcha0[0] + trsp_inv_BT[1][1] * NLcha0[1];

    double nl10 = trsp_inv_BT[0][0] * NLcha1[0] + trsp_inv_BT[0][1] * NLcha1[1];
    double nl11 = trsp_inv_BT[1][0] * NLcha1[0] + trsp_inv_BT[1][1] * NLcha1[1];
```

```
    double nl20 = trsp_inv_BT[0][0] * NLcha2[0] + trsp_inv_BT[0][1] * NLcha2[1];
    double nl21 = trsp_inv_BT[1][0] * NLcha2[0] + trsp_inv_BT[1][1] * NLcha2[1];

    //Calcul des produits scalaires * ET
    double val00 = val * (nl00 * nl00 + nl01 * nl01);
    double val01 = val * (nl00 * nl10 + nl01 * nl11);
    double val02 = val * (nl00 * nl20 + nl01 * nl21);
    double val11 = val * (nl10 * nl10 + nl11 * nl11);
    double val12 = val * (nl10 * nl20 + nl11 * nl21);
    double val22 = val * (nl20 * nl20 + nl21 * nl21);

    return {{val00, val01, val02},
        {val01, val11, val12},
        {val02, val12, val22}
    };
}


matrix ReacTerm(duplix xs_ys){
    vector<double> xs = get<0>(xs_ys);
    vector<double> ys = get<1>(xs_ys);

    double det = (xs[1]-xs[0])*(ys[2]-ys[0]) - (ys[1]-ys[0])*(xs[2]-xs[0]);

    //Calcul des integrales des lambda_i * lambda_j
    double int_lambda_ii = abs(det) * 1.0/12.0;
    double int_lambda_ij = abs(det) * 1.0/24.0;

    return {{int_lambda_ii, int_lambda_ij, int_lambda_ij},
        {int_lambda_ij, int_lambda_ii, int_lambda_ij},
        {int_lambda_ij, int_lambda_ij,int_lambda_ii}};
}




vector<double> matvec(vector<double> V, maillage TRG, int N, int M,
    double a, double b, double (* eta)(double,double)){

    int K = 2*N*M;
    vector<double> W((N+1)*(M+1));

    // Contient les valeurs de l'integrales de eta sur chaque T
    vector<double> VALS = integ_eta_triang(eta,TRG, N,M,a,b);

    for (int t = 0; t < K; t++){
        Triangle T = TRG[t];

```

```
        // Calcul de B_T
        duplix xs_ys = CoordsTrig(a,b,N,M,T);
        matrix D = DiffTerm(xs_ys,VALS[t]);
        matrix R = ReacTerm(xs_ys);

        for (int i = 0; i < 3; i++){
            int k = T.get(i);

            // Calcul de la contribution
            double res = 0;

            for (int r = 0; r < 3; r++){
                int j = T.get(r);

                // aT = Diff + Reac
                double PROD2 = D[r][i]+R[r][i];
                res += V[j] * PROD2;
            }

            W[k] += res;
        }
    }

    return W;
}



vector<double> scdmembre(double rhsf(double,double,int), int N, int M, maillage TRG,
    double a, double b, int m){

    vector<double> B((N+1)*(M+1));
    int K = 2*N*M;

    for (int t = 0; t < K; t++){

        duplix xs_ys = CoordsTrig(a,b,N,M,TRG[t]);
        vector<double> xs = get<0>(xs_ys);
        vector<double> ys = get<1>(xs_ys);

        double det = abs((xs[1]-xs[0])*(ys[2]-ys[0]) - (ys[1]-ys[0])*(xs[2]-xs[0]));

        // calcul des milieux
        double m0x = (xs[0] + xs[1])/2.;
        double m0y = (ys[0] + ys[1])/2.;
        double m1x = (xs[1] + xs[2])/2.;
        double m1y = (ys[1] + ys[2])/2.;
        double m2x = (xs[0] + xs[2])/2.;
        double m2y = (ys[0] + ys[2])/2.;

```

```
        // Calculs de (f \circ F_T)*(w \circ F_T) en les milieux des côtés du triangles T^
        double f0 = rhsf(m0x,m0y,m);
        double f1 = rhsf(m1x,m1y,m);
        double f2 = rhsf(m2x,m2y,m);

        // Calcul des contributions des sommets de T aux noeuds
        B[TRG[t].get(0)] += (det * (f0*0.5 + f2*0.5))/6;
        B[TRG[t].get(1)] += (det * (f0*0.5 + f1*0.5))/6;
        B[TRG[t].get(2)] += (det * (f1*0.5 + f2*0.5))/6;
    }

    return B;
}


double normL2(vector<double> V,maillage TRG, int N, int M, double a, double b){

    int K = 2*N*M;
    int G = (N+1)*(M+1);
    vector<double> W(G);

    // Calcul de AV
    for (int t = 0; t < K; t++){
        Triangle T = TRG[t];

        duplix xs_ys = CoordsTrig(a,b,N,M,T);
        matrix R = ReacTerm(xs_ys);

        for (int i = 0; i < 3; i++){
            int k = T.get(i);

            // Calcul de la contribution
            double res = 0;

            for (int r = 0; r < 3; r++){
                int j = T.get(r);

                // aT = Diff + Reac (mais Diff = 0 car eta = 0)
                res += V[j] * R[r][i];
            }

            W[k] += res;
        }
    }

    // Calcul de V^T * (AV)
    double norme_carre = 0;
    for (int k = 0; k < G; k++)
        norme_carre += V[k]*W[k];

```

```
    return sqrt(norme_carre);
}


double normL2Grad(vector<double> V, maillage TRG, int N, int M, double a,
    double b){

    int K = 2*N*M;
    int G = (N+1)*(M+1);
    vector<double> W(G);

    // Contient les valeurs de l'integrales de eta sur chaque T
    vector<double> VALS = integ_eta_triang([](double,double){return 1.;},TRG, N,M,a,b);

    // Calcul de AV
    for (int t = 0; t < K; t++){
        Triangle T = TRG[t];

        duplix xs_ys = CoordsTrig(a,b,N,M,T);
        matrix D = DiffTerm(xs_ys,VALS[t]);

        for (int i = 0; i < 3; i++){
            int k = T.get(i);

            // Calcul de la contribution
            double res = 0;

            for (int r = 0; r < 3; r++){
                int j = T.get(r);

                // aT = Diff + Reac (mais reac = 0 car lambda = 0)
                res += V[j] * D[r][i];
            }

            W[k] += res;
        }
    }

    // Calcul de V^T * (AV)
    double norme_carre = 0;
    for (int k = 0; k < G; k++)
        norme_carre += V[k]*W[k];

    return sqrt(norme_carre);
}

// Fonction qui calcule le produit scalaire de deux vecteurs
double pdt_sc(vector<double> u, vector<double> v){
    double prod = 0;
    int N = u.size();
```

```

    for (int k = 0; k < N; k++){
        prod += u[k]*v[k];
    }

    return prod;
}

// Fonction qui renvoie la combinaison linéaire de deux vecteurs
vector<double> cl_vec(double lbd, vector<double> u, double mu, vector
    <double> v){
    int L = u.size();
    vector<double> cl(L);

    for (int k = 0; k < L; k++){
        cl[k] = lbd*u[k] + mu*v[k];
    }

    return cl;
}


// Fonction qui renvoie la composante max en valeur absolue d'un
    vecteur
double max_abs(vector<double> V){
    double max = V[0];

    for (double x : V)
        if (abs(x) > max) max = abs(x);

    return max;
}

vector<double> bicg_stab(vector<double> B,maillage TRG, int N, int M,
     double a,
    double b, double tol,int max_it, double (* eta) (double,double)){

    vector<double> AW,AS,S,R_tmp;
    vector<double> X((N+1)*(M+1),0);

    // on choisit X0=0 (ca nous évite un prod mat vec)
    vector<double> R = B;
    vector<double> R_etoile = R;


    vector<double> W = R;
    int it = 0;

    // Itérations, bicg_stab selon l'algorithme donné
    while (max_abs(R) > tol && it < max_it){
        AW = matvec(W,TRG,N,M,a,b,eta);
        double alpha = pdt_sc(R,R_etoile) / pdt_sc(AW,R_etoile);
```

```
        S = cl_vec(1.,R,-alpha,AW);

        AS = matvec(S,TRG,N,M,a,b,eta);
        double omega = pdt_sc(AS,S)/pdt_sc(AS,AS);

        // Actualisation des variables pour le prochain tour de boucle
        X = cl_vec(1.,X,1.,cl_vec(alpha,W,omega,S));
        R_tmp = R;
        R = cl_vec(1.,S,-omega,AS);

        double beta = (pdt_sc(R,R_etoile) * alpha) / (pdt_sc(R_tmp,R_etoile) * omega);
        W = cl_vec(1.,R,beta,cl_vec(1.,W,-omega,AW));
        it++;
    }

    return X;
}


vector<double> erreurs(double (* solExa) (double,double, int),vector<double> uh,
    maillage TRG, int N, int M, double a, double b, int m){

    vector<double> tab_err(3);
    int G = (N+1)*(M+1);

    vector<double> uchap(G);

    for (int s = 0; s < G; s++){
        tuple<int,int> coord_noeud = invnumgb(N,M,s);
        double x = -a + (2*a)/N * get<0>(coord_noeud);
        double y = -b + (2*b)/M * get<1>(coord_noeud);

        uchap[s] = solExa(x,y,m);
    }


    vector<double> diff = cl_vec(1.,uchap,-1.,uh);

    tab_err[0] = normL2(diff,TRG,N,M,a,b) / normL2(uchap,TRG,N,M,a,b);
    tab_err[1] = normL2Grad(diff,TRG,N,M,a,b) / normL2Grad(uchap,TRG,N,M,a,b);
    tab_err[2] = max_abs(diff) / max_abs(uchap);

    return tab_err;
}
```

Chapitre VI.
Annexe C : classe Triangle

Le code ci-dessous permet de créer la classe Triangle, entièrement définie par le numéro de ses sommets dans le maillage. Elle comprend également un getter qui renvoie le numéro du sommet demandé. Nous avons de plus surchargé l'opérateur = afin de faciliter les tests.

triangle.hpp

```cpp
#ifndef TRIANGLE_H
#define TRIANGLE_H

class Triangle {

private:
    int n1, n2, n3;

public:
    Triangle(int n1, int n2, int n3);
    Triangle();
    int get(int k);

    // Surcharge de ==
    bool operator==(const Triangle& autre) const;
};
#endif
```

triangle.cpp

```cpp
#include "triangle.hpp"


Triangle::Triangle(int n1, int n2, int n3){
    this->n1 = n1;
    this->n2 = n2;
    this->n3 = n3;
}
```

```cpp


Triangle::Triangle(){
    this->n1 = 0;
    this->n2 = 0;
    this->n3 = 0;
}


int Triangle::get(int k){
    switch (k){
        case 0 : return this->n1;
        case 1 : return this->n2;
        case 2 : return this-> n3;
        default : return -1;
    }
}

// Fonction qui renvoie le min de 3 entiers
int min3(int a, int b, int c) {
    int m = (a < b) ? a : b;
    return (m < c) ? m : c;
}


// Fonction qui renvoie le max de 3 entiers
int max3(int a, int b, int c) {
    int m = (a > b) ? a : b;
    return (m > c) ? m : c;
}


// Surcharge de l'opérateur == pour définir l'égalité de triangles
bool Triangle::operator==(const Triangle &t) const {
    int min1 = min3(n1,n2,n3);
    int max1 = max3(n1,n2,n3);
    int mid1 = n1 + n2 + n3 - min1 - max1;

    int min2 = min3(t.n1,t.n2,t.n3);
    int max2 = max3(t.n1,t.n2,t.n3);
    int mid2 = t.n1 + t.n2 + t.n3 - min2 - max2;

    return min1==min2 && mid1==mid2 && max1==max2;
}
```

Chapitre VII.
Annexe D : classe ElementVh

Le code ci-dessous permet de créer la classe ElementVh, comme suggéré en remarque en début de partie 3. Elle comprend des fonctions get, permettant de récupérer ses différentes propriétés. Cependant, nous n'avons pas trouvé son utilisation utile dans la façon dont nous avons écrit le programme. En effet, nous aurions pu l'implémenter dans l'algorithme bicg_stab qui aurait renvoyé un ElementVh plutôt que le vecteur<double> X en modifiant la fin de bicg_stab comme suit :

alternative fin bicg_stab

```
vector<double> bicg_stab(vector<double> B,maillage TRG, int N, int M,
     double a,
    double b, double tol,int max_it, double (* eta) (double,double)){
    ...
    ...
    ...
    double norme_L2 = normL2(X, TRG, N, M, a, b);
    double norme_L2_Grad = normL2Grad(X, TRG, N, M, a, b);
    ElementVh solution(X, X, norme_L2, norme_L2_Grad);
return solution;
}
```

Quoiqu'il en soit, nous avons cependant décidé de ne pas l'utiliser et de ne la mettre ici qu'à titre informatif.

elementvh.hpp

```cpp
#ifndef ELEMENTVH_H
#define ELEMENTVH_H

#include <vector>

using namespace std;

class ElementVh {

private:
    vector<double> valeurs_noeuds_interieurs;
    vector<double> valeurs_tous_noeuds;

    double normeL2, normeL2_grad;

public:
    ElementVh(vector<double> valeurs_noeuds_interieurs, vector<double> valeurs_tous_noeuds,
        double normeL2, double normeL2_grad);
    ElementVh();
    vector<double> get_noeuds_interieurs();
    vector<double> get_tous_noeuds();
    double get_norme();
    double get_norme_grad();
};

#endif
```

elementvh.cpp

```cpp
#include "elementvh.hpp"

ElementVh::ElementVh(vector<double> valeurs_noeuds_interieurs, vector<double> valeurs_tous_noeuds,
        double normeL2, double normeL2_grad){
    this->valeurs_noeuds_interieurs = valeurs_noeuds_interieurs;
    this->valeurs_tous_noeuds = valeurs_tous_noeuds;
    this->normeL2 = normeL2;
    this->normeL2_grad = normeL2_grad;
}

ElementVh::ElementVh(){
    this->valeurs_noeuds_interieurs = vector<double>();
    this->valeurs_tous_noeuds = vector<double>();
    this->normeL2 = 0;
    this->normeL2_grad = 0;
}

vector<double> ElementVh::get_noeuds_interieurs(){
    return this->valeurs_noeuds_interieurs;
```

```
}

vector<double> ElementVh::get_tous_noeuds(){
    return this->valeurs_tous_noeuds;
}

double ElementVh::get_norme(){
    return this->normeL2;
}

double ElementVh::get_norme_grad(){
    return this->normeL2_grad;
}
```

Chapitre VIII.
Annexe E : fichier de test

Cette annexe présente, le fichier tests.cpp, qui regroupe l'ensemble des tests effectués tout au long du projet, dans le but de s'assurer de la correction des fonctions implémentées.

tests.cpp

```cpp

#include <cmath>
#include <iostream>

#include "fonctions.hpp"

using namespace std;


// Code ANSI des couleurs
const string VERT = "\033[32m";
const string ROUGE = "\033[31m";
const string BLANC = "\033[0m";


void test_numgb(int &k, int N,int M, int i, int j,int sol_attendue){
    int sol_obtenue = numgb(N,M,i,j);

    if (sol_obtenue == sol_attendue){
        cout << VERT << "Test "<<k<<" : réussi !\n"<< BLANC ;
    } else {
        cout << ROUGE << "Test "<<k<<" : échoué !\n" << BLANC ;
        cout << "   Résultat obtenu : " << sol_obtenue << endl;
        cout << "   Résultat attendu : " << sol_attendue << endl;
    }
    k++;
}


```

```
void test_invnumgb(int &k, int N, int M, int s, tuple<int,int> sol_attendue){
    tuple<int,int> sol_obtenue = invnumgb(N,M,s);

    if (sol_obtenue == sol_attendue){
        cout << VERT << "Test "<<k<<" : réussi !\n" << BLANC;
    } else {
        cout << ROUGE << "Test "<<k<<" : échoué !\n" << BLANC;
        cout << "Résultat obtenu : (" << get<0>(sol_obtenue)<<","<<get<1>(sol_obtenue) << ")\n";
        cout << "Résultat attendu : (" << get<0>(sol_attendue)<<","<<get<1>(sol_attendue) << ")\n";
    }
    k++;
}


void test_maillageTR(int &k, int N, int M, vector<Triangle> sol_attendue){
    vector<Triangle> sol_obtenue = maillageTR(N,M);

    int K = 2*N*M;
    bool reussi = true;

    for (int i = 0; i < K; i++){
        if (!(sol_obtenue[i] == sol_attendue[i])){
            reussi = false;
            break;
        }
    }

    if (reussi){
        cout << VERT << "Test "<<k<<" : réussi !\n" << BLANC;
    } else {
        cout << ROUGE << "Test "<<k<<" : échoué !\n" << BLANC;
        cout << "   Pas d'affichage de debug pour le moment.\n";
    }
    k++;

}


void test_CoordsTrig(int &num_test, double a, double b, int N, int M, Triangle T,
    tuple<vector<double>,vector<double>> sol_attendue){

    auto sol_obtenue = CoordsTrig(a,b,N,M,T);
    int L = get<0>(sol_attendue).size();
    auto xs_attendu = get<0>(sol_attendue);
    auto ys_attendu = get<1>(sol_attendue);
    auto xs_obtenu = get<0>(sol_obtenue);
    auto ys_obtenu = get<1>(sol_obtenue);
```

```
    bool correct_xs = true;
    bool correct_ys = true;

    for (int k = 0; k < L; k++){
        if (correct_xs && xs_attendu[k] != xs_obtenu[k]) correct_xs = false;
        if (correct_ys && ys_attendu[k] != ys_obtenu[k]) correct_ys = false;
    }

    if (correct_xs && correct_ys){
        cout << VERT << "Test "<< num_test<<" : réussi !\n" << BLANC;
    } else {
        cout << ROUGE << "Test "<<num_test<<" : échoué !\n" << BLANC;
        if (correct_ys){
            cout << "Il y a un soucis avec au moins une composante en x.\n";
        }
        if (correct_xs){
            cout << "Il y a un soucis avec au moins une composante en y.\n";
        }
    }

    num_test++;
}


void test_CalcMatBT(int &num_test, vector<double> xs, vector<double> ys,
    matrix sol_attendue){

    auto sol_obtenue = CalcMatBT(xs,ys);
    bool reussi = true;

    for (int l = 0; l < 2; l++){
        for (int c = 0; c < 2; c++){
            if (sol_obtenue[l][c] != sol_attendue[l][c]){
                reussi = false;
                break;
            }
        }
    }

    if (reussi){
        cout << VERT << "Test "<<num_test<<" : réussi !\n" << BLANC;
    } else {
        cout << ROUGE << "Test "<<num_test<<" : échoué !\n" << BLANC;
        cout << "   Pas d'affichage de debug pour le moment.\n";
    }

    num_test++;
```

```
}


void test_integ_eta_triang(int &num_test, double (* eta)(double, double),
    vector<Triangle> maillage,int N, int M,double a,double b,
    vector<double> sol_attendue){

    int K = 2*N*M;
    bool reussi = true;
    vector<double> sol_obtenue = integ_eta_triang(eta,maillage,N,M,a,b);
    double epsilon = 1e-9;

    for (int k = 0; k < K; k++){
        if (abs(sol_attendue[k] - sol_obtenue[k]) > epsilon){
            reussi = false;

            break;
        }
    }

    if (reussi){
        cout << VERT << "Test "<<num_test<<" : réussi !\n" << BLANC;
    } else {
        cout << ROUGE << "Test "<<num_test<<" : échoué !\n" << BLANC;
        cout << "   Pas d'affichage de debug pour le moment.\n";
    }

    num_test++;
}


void test_DiffTerm(int &num_test,duplix xs_ys, double val, matrix sol_attendue){

    matrix sol_obtenue = DiffTerm(xs_ys,val);
    bool reussi = true;
    double epsilon = 0.0000001;

    for (int l = 0; l < 3; l++){
        for (int c = 0; c < 3; c++){
            if (abs(sol_obtenue[l][c] - sol_attendue[l][c]) > epsilon){
                reussi = false;
                break;
            }
        }
    }

    if (reussi){
        cout << VERT << "Test "<<num_test<<" : réussi !\n" << BLANC;
```

```
    } else {
        cout << ROUGE << "Test "<<num_test<<" : échoué !\n" << BLANC;
        cout << "   Pas d'affichage de debug pour le moment.\n";
    }

    num_test++;
}


void test_ReacTerm(int &num_test,duplix xs_ys, matrix sol_attendue){

    matrix sol_obtenue = ReacTerm(xs_ys);
    bool reussi = true;
    double epsilon = 0.0000001;

    for (int l = 0; l < 3; l++){
        for (int c = 0; c < 3; c++){
            if (abs(sol_obtenue[l][c] - sol_attendue[l][c]) > epsilon
    ){
                reussi = false;
                break;
            }
        }
    }

    if (reussi){
        cout << VERT << "Test "<<num_test<<" : réussi !\n" << BLANC;
    } else {
        cout << ROUGE << "Test "<<num_test<<" : échoué !\n" << BLANC;
        cout << "   Pas d'affichage de debug pour le moment.\n";
    }

    num_test++;
}


void test_matvec(int &num_test, vector<double> V, vector<Triangle>
    maillage,
    int N, int M, double a, double b, double (* eta)(double,double),
    vector<double> sol_attendue){


    vector<double> sol_obtenue = matvec(V,maillage,N,M,a,b,eta);
    bool reussi = true;
    double epsilon = 0.0000001;
    int G = (N+1)*(M+1);

    for (int k = 0; k < G; k++){
        if (abs(sol_attendue[k] - sol_obtenue[k]) > epsilon){
            reussi = false;
            break;
        }
```

```
    }

    if (reussi){
        cout << VERT << "Test "<<num_test<<" : réussi !\n" << BLANC;
    } else {
        cout << ROUGE << "Test "<<num_test<<" : échoué !\n" << BLANC;
        cout << "   Pas d'affichage de debug pour le moment.\n";
    }

    num_test++;
}


void test_pdt_sc(int &num_test, vector<double> u, vector<double> v,
        double sol_attendue){

    if (pdt_sc(u,v) == sol_attendue){
        cout << VERT << "Test "<<num_test<<" : réussi !\n" << BLANC;
    } else {
        cout << ROUGE << "Test "<<num_test<<" : échoué !\n" << BLANC;
        cout << "   Pas d'affichage de debug pour le moment.\n";
    }
    num_test++;
}


void test_cl_vec(int &num_test,double lbd, vector<double> u, double
    mu,
    vector<double> v, vector<double> sol_attendue){

    vector<double> sol_obtenue = cl_vec(lbd,u,mu,v);
    bool reussi = true;
    int K = sol_attendue.size();
    double epsilon = 1e-9;

    for (int k = 0; k < K; k++){
        if (abs(sol_attendue[k] - sol_obtenue[k]) > epsilon){
            reussi = false;
            break;
        }
    }

    if (reussi){
        cout << VERT << "Test "<<num_test<<" : réussi !\n" << BLANC;
    } else {
        cout << ROUGE << "Test "<<num_test<<" : échoué !\n" << BLANC;
        cout << "   Pas d'affichage de debug pour le moment.\n";
    }

    num_test++;
}

```

```
void test_max_abs(int &num_test, vector<double> V, double sol_attendue){

    if (max_abs(V) == sol_attendue){
        cout << VERT << "Test "<<num_test<<" : réussi !\n" << BLANC;
    } else {
        cout << ROUGE << "Test "<<num_test<<" : échoué !\n" << BLANC;
        cout << "   Pas d'affichage de debug pour le moment.\n";
    }

    num_test++;
}


void test_scdmembre(int &num_test, double rhsf(double,double,int), int N, int M, maillage TRG, double a, double b, int m, vector<double> sol_attendue){


    vector<double> sol_obtenue = scdmembre(rhsf,N,M,TRG,a,b,m);
    bool reussi = true;
    int K = sol_attendue.size();
    double epsilon = 1e-9;

    for (int k = 0; k < K; k++){
        if (abs(sol_attendue[k] - sol_obtenue[k]) > epsilon){
            reussi = false;
            break;
        }
    }

    if (reussi){
        cout << VERT << "Test "<<num_test<<" : réussi !\n" << BLANC;
    } else {
        cout << ROUGE << "Test "<<num_test<<" : échoué !\n" << BLANC;
        cout << "   Pas d'affichage de debug pour le moment.\n";
    }

    num_test++;
}


void test_normL2(int &num_test,vector<double> V,maillage TRG, int N, int M, double a,
    double b, double sol_attendue){

    if (abs(normL2(V,TRG,N,M,a,b) - sol_attendue) < 1e-9){
        cout << VERT << "Test "<<num_test<<" : réussi !\n" << BLANC;
    } else {
        cout << ROUGE << "Test "<<num_test<<" : échoué !\n" << BLANC;
        cout << "   Pas d'affichage de debug pour le moment.\n";
```

```
    }

    num_test++;
}


void test_normL2Grad(int &num_test, vector<double> V, maillage TRG, int N, int M,
    double a, double b, double sol_attendue){

    if (abs(normL2Grad(V,TRG,N,M,a,b) - sol_attendue) < 1e-9){
        cout << VERT << "Test "<<num_test<<" : réussi !\n" << BLANC;
    } else {
        cout << ROUGE << "Test "<<num_test<<" : échoué !\n" << BLANC;
        cout << "   Pas d'affichage de debug pour le moment.\n";
    }

    num_test++;
}



void test_bicg_stab(int &num_test, vector<double> B,maillage TRG, int N, int M,
    double a, double b, double tol,int max_it, double (* eta) (double,double),
    vector<double> sol_attendue){

    vector<double> sol_obtenue = bicg_stab(B,TRG,N,M,a,b,tol,max_it,eta);
    bool reussi = true;
    int K = sol_attendue.size();
    double epsilon = 1e-9;

    for (int k = 0; k < K; k++){
        if (abs(sol_attendue[k] - sol_obtenue[k]) > epsilon){
            reussi = false;
            break;
        }
    }

    if (reussi){
        cout << VERT << "Test "<<num_test<<" : réussi !\n" << BLANC;
    } else {
        cout << ROUGE << "Test "<<num_test<<" : échoué !\n" << BLANC;
        cout << "   Pas d'affichage de debug pour le moment.\n";
    }

    num_test++;
}


```

```
double eta1(double x, double y) {
    return 1.0;
}

double eta2(double x, double y) {
    return x + 2.0 * y + 5.0;
}

double eta3(double x, double y) {
    return x * x + y * y + 1.0;
}

//fonction qui affiche un vector de double
void print_vector(vector<double> tab){
    cout << "[" ;
    int size = tab.size();
    for (int k = 0; k < size -1; k++) cout << tab[k] << ", ";
    cout << tab[size-1] << "]\n";
}



int main(){
    cout << "--------------Tests numgb--------------\n";

    int num_tests = 1;

    // Test 1,2
    test_numgb(num_tests,2, 2, 1, 1,4);
    test_numgb(num_tests,18, 50, 11, 31,600);


    cout << "\n\n\n";
    cout << "--------------Tests invnumgb--------------\n";

    // Test 3,4,5
    test_invnumgb(num_tests,2, 2,4,{1,1});
    test_invnumgb(num_tests,21, 3,45,{1,2});
    test_invnumgb(num_tests,101, 101,3923,{47,38});



    cout << "\n\n\n";
    cout << "--------------Tests maillageTR--------------\n";



    // Test 6 : N = 1, M = 1
    maillage maillage0(2);

    maillage0[0] = Triangle(0, 2, 3);
    maillage0[1] = Triangle(0, 1, 3);
```

```
    test_maillageTR(num_tests, 1, 1, maillage0);



    // Test 7 : N = 2, M = 1
    maillage maillage1(4);

    maillage1[0] = Triangle(0,3,4);
    maillage1[1] = Triangle(0,1,4);
    maillage1[2] = Triangle(1,2,4);
    maillage1[3] = Triangle(2,4,5);

    test_maillageTR(num_tests,2,1,maillage1);



    // Test 8: N = 3, M = 2
    vector<Triangle> maillage2(12);

    maillage2[0] = Triangle(0, 4, 5);
    maillage2[1] = Triangle(0, 1, 5);
    maillage2[2] = Triangle(1, 2, 5);
    maillage2[3] = Triangle(2, 5, 6);
    maillage2[4] = Triangle(2, 6, 7);
    maillage2[5] = Triangle(2, 3, 7);

    maillage2[6] = Triangle(4, 5, 8);
    maillage2[7] = Triangle(5, 8, 9);
    maillage2[8] = Triangle(5, 9, 10);
    maillage2[9] = Triangle(5, 6, 10);
    maillage2[10] = Triangle(6, 7, 10);
    maillage2[11] = Triangle(7, 10, 11);

    test_maillageTR(num_tests, 3, 2, maillage2);









    // Test 9 : N = 4, M = 2
    vector<Triangle> maillage3 = vector<Triangle>(16);

    maillage3[0] = Triangle(0, 5, 6);
    maillage3[1] = Triangle(0, 1, 6);
    maillage3[2] = Triangle(1, 6, 2);
    maillage3[3] = Triangle(2, 6, 7);
    maillage3[4] = Triangle(2, 7, 8);
    maillage3[5] = Triangle(2, 3, 8);
    maillage3[6] = Triangle(3, 4, 8);
```

```
    maillage3[7] = Triangle(4, 8, 9);

    maillage3[8] = Triangle(5, 6, 10);
    maillage3[9] = Triangle(6, 10, 11);
    maillage3[10] = Triangle(6, 11, 12);
    maillage3[11] = Triangle(6, 7, 12);
    maillage3[12] = Triangle(7, 8, 12);
    maillage3[13] = Triangle(8, 12, 13);
    maillage3[14] = Triangle(8, 13, 14);
    maillage3[15] = Triangle(8, 9, 14);

    test_maillageTR(num_tests,4,2,maillage3);



    // Test 10 : N = 4, M = 4
    vector<Triangle> maillage4(32);

    maillage4[0]  = Triangle(0, 5, 6);
    maillage4[1]  = Triangle(0, 1, 6);
    maillage4[2]  = Triangle(1, 2, 6);
    maillage4[3]  = Triangle(2, 6, 7);
    maillage4[4]  = Triangle(2, 7, 8);
    maillage4[5]  = Triangle(2, 3, 8);
    maillage4[6]  = Triangle(3, 4, 8);
    maillage4[7]  = Triangle(4, 8, 9);

    maillage4[8]  = Triangle(5, 6, 10);
    maillage4[9]  = Triangle(6, 10, 11);
    maillage4[10] = Triangle(6, 11, 12);
    maillage4[11] = Triangle(6, 7, 12);
    maillage4[12] = Triangle(7, 8, 12);
    maillage4[13] = Triangle(8, 12, 13);
    maillage4[14] = Triangle(8, 13, 14);
    maillage4[15] = Triangle(8, 9, 14);

    maillage4[16] = Triangle(10, 15, 16);
    maillage4[17] = Triangle(10, 11, 16);
    maillage4[18] = Triangle(11, 12, 16);
    maillage4[19] = Triangle(12, 16, 17);
    maillage4[20] = Triangle(12, 17, 18);
    maillage4[21] = Triangle(12, 13, 18);
    maillage4[22] = Triangle(13, 14, 18);
    maillage4[23] = Triangle(14, 18, 19);

    maillage4[24] = Triangle(15, 16, 20);
    maillage4[25] = Triangle(16, 20, 21);
    maillage4[26] = Triangle(16, 21, 22);
    maillage4[27] = Triangle(16, 17, 22);
    maillage4[28] = Triangle(17, 18, 22);
    maillage4[29] = Triangle(18, 22, 23);
    maillage4[30] = Triangle(18, 23, 24);
```

```
    maillage4[31] = Triangle(18, 19, 24);

    test_maillageTR(num_tests, 4, 4, maillage4);


    // Test 11 : N = 3, M = 3
    vector<Triangle> maillage5(18);

    maillage5[0] = Triangle(0, 4, 5);
    maillage5[1] = Triangle(0, 1, 5);
    maillage5[2] = Triangle(1, 2, 5);
    maillage5[3] = Triangle(2, 5, 6);
    maillage5[4] = Triangle(2, 6, 7);
    maillage5[5] = Triangle(2, 3, 7);

    maillage5[6] = Triangle(4, 5, 8);
    maillage5[7] = Triangle(5, 8, 9);
    maillage5[8] = Triangle(5, 9, 10);
    maillage5[9] = Triangle(5, 6, 10);
    maillage5[10] = Triangle(6, 7, 10);
    maillage5[11] = Triangle(7, 10, 11);

    maillage5[12] = Triangle(8, 12, 13);
    maillage5[13] = Triangle(8, 9, 13);
    maillage5[14] = Triangle(9, 10, 13);
    maillage5[15] = Triangle(10, 13, 14);
    maillage5[16] = Triangle(10, 14, 15);
    maillage5[17] = Triangle(10, 11, 15);

    test_maillageTR(num_tests, 3, 3, maillage5);


    cout << "\n\n\n";
    cout << "--------------Tests CalcMatBT--------------\n";


    // Test 12
    vector<double> xs1 = {0.0, 1.0, 0.0};
    vector<double> ys1 = {0.0, 0.0, 1.0};
    matrix BT1 = {{1.0, 0.0},{0.0, 1.0}};
    test_CalcMatBT(num_tests, xs1, ys1, BT1);


    // Test 13
    vector<double> xs2 = {1.0, 4.0, 2.0};
    vector<double> ys2 = {1.0, 2.0, 5.0};
    matrix BT2 = {{3.0, 1.0},{1.0, 4.0}};
    test_CalcMatBT(num_tests, xs2, ys2, BT2);


    // Test 14
    vector<double> xs3 = {1.0, 2.0, 3.0};
```

```
    vector<double> ys3 = {1.0, 2.0, 3.0};
    matrix BT3 = {{1.0, 2.0},{1.0, 2.0}};
    test_CalcMatBT(num_tests, xs3, ys3, BT3);



    cout << "\n\n\n";
    cout << "--------------Tests integ_eta_triang--------------\n";


    // Test 15 : eta (x,y) = 1
    int N1 = 2;
    int M1 = 2;

    maillage maillage2x2 =   maillageTR(N1,M1);
    vector<double> sol_attendue1(2*N1*M1, 0.5);

    test_integ_eta_triang(num_tests, eta1, maillage2x2, N1, M1, 1.0, 1.0, sol_attendue1);


    // Test 16 : eta(x,y) = x + 2y + 5
    maillage maillage1x1 =   maillageTR(1,1);
    vector<double> sol_attendue2 = {32./3., 28./3.};
    test_integ_eta_triang(num_tests, eta2, maillage1x1, 1, 1, 1.0, 1.0, sol_attendue2);


    // Test 17 : eta(x,y) = x   + y   + 1
    vector<double> sol_attendue3 = {10.0 / 3.0, 10.0 / 3.0};

    test_integ_eta_triang(num_tests, eta3, maillage1x1, 1, 1, 1.0, 1.0, sol_attendue3);


    cout << "\n\n\n";
    cout << "--------------Tests DiffTerm--------------\n";


    // Test 18
    vector<double> xs4 = {0.0, 1.0, 0.0};
    vector<double> ys4 = {0.0, 0.0, 1.0};
    duplix T1 = {xs4, ys4};
    double val1 = 0.5;

    matrix sol_attendue4 =  {
        { 1.0, -0.5, -0.5},
        {-0.5,  0.5,  0.0},
        {-0.5,  0.0,  0.5}
    };

    test_DiffTerm(num_tests, T1, val1, sol_attendue4);
```

```
626
627
628     // Test 19
629     vector<double> xs5 = {0.0, 2.0, 1.0};
630     vector<double> ys5 = {0.0, 0.0, 1.0};
631     duplix T2 = {xs5, ys5};
632     double val2 = 1.0;
633
634     matrix sol_attendue5 = {
635         { 0.5,  0.0, -0.5},
636         { 0.0,  0.5, -0.5},
637         {-0.5, -0.5,  1.0}
638     };
639
640     test_DiffTerm(num_tests, T2, val2, sol_attendue5);
641
642
643     // Test 20
644     vector<double> xs6 = {1.0, 3.0, 1.0};
645     vector<double> ys6 = {1.0, 1.0, 4.0};
646     duplix T3 = {xs6, ys6};
647     double val3 = 3.0;
648
649     matrix sol_attendue6 = {
650         { 13.0/12.0, -0.75, -1.0/3.0 },
651         { -0.75,      0.75,  0.0 },
652         { -1.0/3.0,   0.0,   1.0/3.0 }
653     };
654
655     test_DiffTerm(num_tests, T3, val3, sol_attendue6);
656
657
658
659     cout << "\n\n\n";
660     cout << "--------------Tests ReacTerm--------------\n";
661
662
663     // Test 21 : triangle de référence
664     vector<double> xs7 = {0.0, 1.0, 0.0};
665     vector<double> ys7 = {0.0, 0.0, 1.0};
666     duplix T4 = {xs7, ys7};
667
668     matrix sol_attendue7 = {
669         { 1.0/12.0, 1.0/24.0, 1.0/24.0 },
670         { 1.0/24.0, 1.0/12.0, 1.0/24.0 },
671         { 1.0/24.0, 1.0/24.0, 1.0/12.0 }
672     };
673
674     test_ReacTerm(num_tests, T4, sol_attendue7);
675
676
677     // Test 22
```

```
    vector<double> xs8 = {1.0, 3.0, 1.0};
    vector<double> ys8 = {1.0, 1.0, 4.0};
    duplix T5 = {xs8, ys8};

    matrix sol_attendue8 = {
        { 0.50, 0.25, 0.25 },
        { 0.25, 0.50, 0.25 },
        { 0.25, 0.25, 0.50 }
    };

    test_ReacTerm(num_tests, T5, sol_attendue8);


    // Test 23
    vector<double> xs9 = {-100.0, 500.0, 0.0};
    vector<double> ys9 = {-50.0, -50.0, 1000.0};
    duplix T6 = {xs9, ys9};

    matrix sol_attendue9 = {
        { 52500.0, 26250.0, 26250.0 },
        { 26250.0, 52500.0, 26250.0 },
        { 26250.0, 26250.0, 52500.0 }
    };

    test_ReacTerm(num_tests, T6, sol_attendue9);


    cout << "\n\n\n";
    cout << "--------------Tests matvec--------------\n";

    // Test 24
    vector<double> V = {0.0, 0.0, 0.0, 0.0};
    vector<double> sol_attendue10 = {0.0, 0.0, 0.0, 0.0};
    test_matvec(num_tests, V, maillage1x1, 1,1,1.0,1.0,eta1,
    sol_attendue10);


    // Test 25
    vector<double> V1 = {1.0, 1.0, 1.0, 1.0};
    vector<double> sol_attendue11 = {4.0 / 3.0, 2.0 / 3.0, 2.0 / 3.0,
     4.0 / 3.0};
    test_matvec(num_tests, V1, maillage1x1, 1, 1, 1.0, 1.0, eta1,
    sol_attendue11);

    // Test 26
    vector<double> V2 = {1.0, 0.0, 0.0, 0.0};
    vector<double> sol_attendue12 = {5.0 / 3.0, -1.0 / 3.0, -1.0 /
    3.0, 1.0 / 3.0};
    test_matvec(num_tests, V2, maillage1x1, 1, 1, 1.0, 1.0, eta1,
    sol_attendue12);


```

```
    cout << "\n\n\n";
    cout << "--------------Tests pdt_sc--------------\n";

    // Test 27
    vector<double> U3 = {1.0, 2.0, 3.0};
    vector<double> V3 = {4.0, -5.0, 6.0};
    test_pdt_sc(num_tests, U3, V3, 12.0);


    // Test 28
    vector<double> U4 = {1.0, 0.0, -1.0};
    vector<double> V4 = {0.0, 1.0, 0.0};
    test_pdt_sc(num_tests, U4, V4, 0.0);

    // Test 29
    vector<double> U5 = {0.0, 0.0, 0.0};
    vector<double> V5 = {7.5, -3.14, 42.0};
    test_pdt_sc(num_tests, U5, V5, 0.0);

    // Test 30
    vector<double> U6 = {0.5, 0.25, 1.5};
    vector<double> V6 = {2.0, 4.0, -2.0};
    test_pdt_sc(num_tests, U6, V6, -1.0);


    cout << "\n\n\n";
    cout << "--------------Tests cl_vec--------------\n";

    // Test 31
    vector<double> U7 = {1.0, 2.0, 3.0};
    vector<double> V7 = {4.0, 5.0, 6.0};
    vector<double> sol_attendue13 = {14.0, 19.0, 24.0};
    test_cl_vec(num_tests, 2.0, U7, 3.0, V7, sol_attendue13);

    // Test 32
    vector<double> U8 {1.5, -2.5};
    vector<double> V8 = {1.5, -2.5};
    vector<double> sol_attendue14 = {0.0, 0.0};
    test_cl_vec(num_tests, 1.0, U8, -1.0, V8, sol_attendue14);

    // Test 33
    vector<double> U9 = {10.0, 20.0};
    vector<double> V9 = {30.0, 40.0};
    vector<double> sol_attendue15= {0.0, 0.0};
    test_cl_vec(num_tests, 0.0, U9, 0.0, V9, sol_attendue15);


    // Test 34
    vector<double> U10 = {1.0, 3.0};
    vector<double> V10 = {4.0, -8.0};
    vector<double> sol_attendue16 = {1.5, -0.5};
    test_cl_vec(num_tests, 0.5, U10, 0.25, V10, sol_attendue16);
```

```
    cout << "\n\n\n";
    cout << "--------------Tests max_abs--------------\n";

    // Test 35
    vector<double> V11 = {1.5, -2.0, 3.5, -1.0};
    test_max_abs(num_tests, V11, 3.5);

    // Test 36
    vector<double> V12 = {4.0, -7.2, 3.1, 5.0};
    test_max_abs(num_tests, V12, 7.2);

    // Test 37
    vector<double> V13 = {0.0, 0.0, 0.0};
    test_max_abs(num_tests, V13, 0.0);



    cout << "\n\n\n";
    cout << "--------------Tests scdmembre--------------\n";



    // Test 38
    vector<double> sol_attendue17(11*15, 0.0);
    auto f0 = [](double,double,int){return 0.;};
    maillage maillage10x14 = maillageTR(10,14);
    test_scdmembre(num_tests, f0 , 10, 14, maillage10x14, 1., 1., 1, sol_attendue17);



    // Test 39
    auto f1 = [](double,double,int){return 1.0;};
    double aire_T = (2.0 * 2.0) / 6.0;
    vector<double> sol_attendue18 = {2*aire_T,aire_T,aire_T,2*aire_T};
    test_scdmembre(num_tests, f1, 1, 1, maillage1x1, 1.0, 1.0, 1, sol_attendue18);



    cout << "\n\n\n";
    cout << "--------------Tests normL2--------------\n";

    int N2 = 2, M2 = 2;
    double a = 1.0, b = 1.0;
    int G = (N2 + 1) * (M2 + 1);
```

```
    // Test 40
    vector<double> V14(G, 0.0);
    test_normL2(num_tests, V14, maillage2x2, N2, M2, a, b, 0.0);

    // Test 41
    vector<double> V15(G, 1.0);
    test_normL2(num_tests, V15, maillage2x2, N2, M2, a, b, 2.0);

    // Test42
    vector<double> V16(G, 0.0);
    for (int s = 0; s < G; s++) {
        tuple<int,int> coord = invnumgb(N2, M2, s);
        V16[s] = -a + (2.0 * a) / N2 * get<0>(coord);
    }

    test_normL2(num_tests, V16, maillage2x2, N2, M2, a, b, sqrt(4.0 /
     3.0));


    cout << "\n\n\n";
    cout << "--------------Tests normL2Grad--------------\n";


    // Test 43
    test_normL2Grad(num_tests, V15, maillage2x2, N2, M2, a, b, 0.0);

    // Test 44
    test_normL2Grad(num_tests, V16, maillage2x2, N2, M2, a, b, 2.0);

    vector<double> V17(G, 0.0);
    for (int s = 0; s < G; s++) {
        tuple<int,int> coord = invnumgb(N2, M2, s);
        double x_noeud = -a + (2.0 * a) / N2 * get<0>(coord);
        double y_noeud = -b + (2.0 * b) / M2 * get<1>(coord);
        V17[s] = x_noeud + y_noeud;
    }

    // Test 45
    test_normL2Grad(num_tests, V17, maillage2x2, N2, M2, a, b, sqrt
    (8.0));



    cout << "\n\n\n";
    cout << "--------------Test bicg_stab--------------\n";


    // Test 46
    int N3 = 2, M3 = 2;
    double a3 = 1.0, b3 = 1.0;
```

```
    int G3 = (N3 + 1) * (M3 + 1);

    vector<double> sol_exacte = {8.,-3.,6.,-9.81,3.14,2.72,7.1,4.2,-5.3};


    vector<double> B = matvec(sol_exacte, maillage2x2, N3, M3, a3, b3, eta1);

    double tol = 1e-10;
    int max_it = 2000;


    test_bicg_stab(num_tests, B, maillage2x2, N3, M3, a3, b3, tol, max_it, eta1, sol_exacte);

}
```